A Simple Batched Threshold Encryption Scheme

Guru-Vamsi Policharla
Commonware, Inc.

Abstract

Batched threshold encryption allows any t-out-of-N parties in a committee to decrypt a batch of B ciphertexts using sub-linear $o(NB)$ communication, while ensuring that any subset of $< t$ colluding parties learns no information about the underlying plaintext.

Our first result is a simple batched threshold encryption scheme that is censorship resistant, avoids epoch restrictions, and achieves quasi-linear $O(B \log B)$ decryption complexity in the batch size B. Our scheme has the shortest ciphertext among all known constructions: $|\mathbb{G}_1| + |\mathbb{G}_T|$ for CPA security, with CCA security adding only $2|\mathbb{F}|$ via a simulation-extractable NIZK. However, our scheme requires an interactive setup phase (involving secure multiplications) and secret keys held by the committee grow linearly with the batch size. We prove security under the Decisional Bilinear B-Power Diffie-Hellman assumption in asymmetric pairing groups and provide an implementation in Rust to show that our scheme outperforms prior work.

We also construct a variant of simple BTE, which allows for a tradeoff between secret-key size and censorship resistance. For any $\delta \leq B$, this variant reduces the size of the secret key held by each party from $O(B) \to O(\delta)$, and the decryption key from $O(BN) \to O(B + \delta N)$, but this comes at the cost of restricting a ciphertext encrypted with index idx to batch positions $i \in [B]$ satisfying $|i - \mathsf{idx}| < \delta$.

Our second result is a new approach for *verifying* decryption in batched threshold encryption which enables a helper party (that carries out decryption) to provide hints that allow a verifier to check that decryption was carried out correctly using only MSMs and hashes. Concretely, we observe a $114.1\times$ speedup when verifying decryption of 2048 ciphertexts when compared against local decryption. Our approach is quite general and can be applied to other pairing-based advanced encryption schemes such as Timelock Encryption and Silent Threshold Encryption that can be cast as witness encryption schemes.

1 Introduction

A threshold encryption scheme [RSA78, ElG86, DF90, Pai99, FPS01, DJ01, GKPW24] is a cryptographic primitive that allows users to encrypt messages to a committee of N parties such that any t-out-of-N members can *non-interactively* decrypt the message.

A prominent application of threshold encryption is in building encrypted mempools, to protect the privacy of "pending" transactions in blockchains [BO22, MS22, KLJD23, CGPP24]. Here, users encrypt their transactions to a validator set, that waits to finalize ordering of the transactions before decryption. Although the above approach works in theory, the communication required to decrypt transactions is prohibitively expensive. Using a "standard" threshold encryption scheme such as ElGamal [ElG86] requires $O(NB)$ communication to broadcast the decryption shares to the network. This can be orders of magnitude larger than the block itself!

Batched Threshold Encryption (BTE) was introduced in [CGPP24] to address this very issue. It allows the committee to decrypt a batch of B ciphertexts using sub-linear communication $o(NB)$. The main limitation of the initial construction was that it required an interactive setup phase (involving secure multiplications) i) at the beginning of the protocol and ii) for every batch of ciphertexts being decrypted. Since then, a long line of follow-up work [CGPW25, AFP25, SAS24, BFOQ25, BLT25, FPTX25, ABD+25, BCF+25, GWWW25, BNRT26] has pushed the primitive closer to practicality, albeit with caveats of their own that we expand on below and summarize in Table 1.

Epoch Restriction. [CGPW25, AFP25] were essentially the same construction but with a slightly different formalization. Both works showed how to build a BTE scheme with a one-time DKG setup but ciphertexts still had to be encrypted to a certain batch number (epoch). Failure to be included in the chosen epoch would result in the ciphertext never being decrypted. [AFP25] additionally observed that the construction satisfies a stronger notion of Batched Identity Based Encryption where users can encrypt to arbitrary identities just given the master public key, but issuing keys for a batch of B identities can be done using sub-linear communication. [GWWW25] showed that a modification of [CGPW25, AFP25] is a secure Batched IBE scheme under a falsifiable assumption in the non-threshold setting. They also gave an alternate construction in the Generic Group Model [Sho97, Mau05] that supports threshold decryption. Also worth noting are [BCF+25] and [GWWW25] that build batched threshold encryption and batched identity based encryption respectively, with silent setup [GKPW24, GHK+25].

[FPTX25] partially addressed the epoch restriction but at the cost of a k-fold increase in CRS size – which allows a user's ciphertext to be included in any of the k batches. However, at the end of k batches, the public key must be refreshed (which requires a DKG). Furthermore, censorship resistance comes at the cost of a more expensive $O(B \log^2 B)$ decryption complexity.

Censorship Issues. We define censorship resistance as the minimum number of ciphertexts that an adversary can include in a batch before a victim's ciphertext cannot be included in the batch.[1] Ideally, this is the same as the maximum batch size B, i.e., to prevent a ciphertext from being included in the batch, an attacker must fill up the entire block which translates to buying up the entire block.

Prior work [BFOQ25, ABD+25] avoid epoch restriction but suffered from censorship issues where an attacker could block a victim's ciphertext by including a *single* ciphertext encrypted to a conflicting position. To rectify this:

- [BFOQ25] offer "sub-batching" as a solution where the ciphertexts are divided into non-conflicting sub-batches which are decrypted separately. However, this increases the size of partial decryptions and in the worst case may result in B sub-batches each containing a single ciphertext – defeating the purpose of batched threshold encryption.
- [ABD+25] uses a different approach where users encrypt to *multiple* positions (δ say) in the batch (chosen via cuckoo hashing). The ciphertext size grows as $O(\delta)$, but this increases the censorship resistance to δ, as the adversary must fill up all δ positions to block a ciphertext.

Large Secret Keys. Finally, [BNRT26] provided a construction based on partial fraction techniques [JNR25] which avoids both the epoch restriction and censorship issues. However, this comes at the cost of an interactive setup (involving secure inversions) and the size of secret keys held by each party grows with the batch size B.

In the context of encrypted mempools, epoch restrictions and censorship issues affect user experience as they have to maneuver around *clunky* cryptography artifacts if they want good inclusion guarantees. This discourages adoption as users are forced to make tradeoffs between privacy and usability. When rotating committees, large secret keys pose a problem as either the setup needs to be repeated (in which case user's wallets would need to download the latest public key) or the secret keys are securely re-shared to the new committee. However, this is a fixed, predictable cost that is only paid if/when the validator set is rotated. This is strongly preferable to the alternative of an inconvenience to the end user in every transaction.

1.1 Our Contributions

Simple BTE. We construct a simple batched threshold encryption scheme (Section 4) that simultaneously achieves censorship resistance, avoids epoch restrictions, and supports quasi-linear $O(B \log B)$ decryption

[1] Note that these ciphertexts may not be efficiently computable in practice. We use the "information-theoretic" definition to simplify the exposition and avoid introducing additional hardness assumptions.

complexity. Like [BNRT26], our scheme accepts an interactive setup phase and secret keys that grow with the batch size (see Table 1 for a detailed comparison).

Our CPA ciphertext size is $|\mathbb{G}_1| + |\mathbb{G}_T|$, which is shorter than all prior BTE constructions, which require at least $2|\mathbb{G}_1| + |\mathbb{G}_T|$. CCA security is obtained by appending a simulation-extractable NIZK proof of well-formedness, adding $2|\mathbb{F}|$ to the ciphertext. Naively, decryption takes $O(B^2)$ pairings, but we show how to accelerate it to $O(B \log B)$ using FFT-based techniques (Section 6.1). Furthermore, we observe that the expensive FFT computation depends only on the ciphertexts and public parameters, not on the secret key material. This allows the bulk of the decryption work to be pipelined concurrently with the threshold decryption protocol, so that once the partial decryptions are received only B additional pairings are needed to recover the plaintexts (Section 6.2). In practice, this reduces the post-protocol latency by $7.5\times$–$12.3\times$ depending on batch size: e.g., at $B = 2048$ the computation after the threshold shares are received drops from 12.91 s to just 1.05 s. We prove security under the Decisional Bilinear B-Power Diffie-Hellman assumption [BGW05] in asymmetric pairing groups.

Censorship vs. $|\mathsf{sk}|$. While simple BTE achieves maximum censorship resistance, rotating committees requires either re-sharing $O(B)$ secrets or re-doing an interactive setup involving $O(B)$ secure multiplications, both of which are quite expensive. To address this, we modify the scheme to allow for a tradeoff between secret-key size and censorship resistance at the expense of a slightly larger ciphertext. Concretely, for any $\delta \leq B$, if the committee holds shares of $O(\delta)$ secrets, the decryption key reduces from $O(BN) \to O(B+\delta N)$, but the censorship resistance reduces to δ, and the ciphertext size grows to $2|\mathbb{G}_1| + |\mathbb{G}_T|$. Although the setup still requires $O(B)$ secure multiplications, the committee only needs to re-share $O(\delta)$ secrets to rotate committees. A formal description of the construction is provided in Section 5.

Interestingly, at $\delta = 1$ (censorship resistance drops to 1), this is the first scheme without epoch restrictions to simultaneously have an $O(B+N)$ decryption key and $2|\mathbb{G}_1| + |\mathbb{G}_T|$ ciphertext size. Prior work either required a large decryption key $O(BN)$ to achieve the same ciphertext size [ABD+25], had an $O(B)$ decryption key but a larger ciphertext size $3|\mathbb{G}_1| + |\mathbb{G}_T|$ [BFOQ25], suffered from epoch restrictions [GWWW25], or required a public-key refresh (via a DKG) after every batch when instantiated with an $O(B)$ decryption key [FPTX25].

Bandwidth vs. Computation. Even though many of the above constructions support quasi-linear decryption complexity which is nearly optimal, decryption is lower bounded by ≈ 1 ms *per ciphertext* simply because all schemes (including ours) require at least a few pairings per ciphertext. Validators would need to dedicate over 200 threads to support the 200K TPS needed on performant decentralized exchanges.[2] Unfortunately, designing a batched threshold encryption scheme that avoids pairings appears to be quite challenging. In fact for some approaches [CGPW25, AFP25, FPTX25], there are closely related lower bounds [BPR+08, PRV12] suggesting that a blackbox pairing-free construction is impossible.

Instead, we turn to new system architectures that can help us achieve our goal of high-throughput encrypted mempools. While decryption cannot avoid pairings, we show in Section 7 that it is possible to *verify decryption* much faster by using "hints" that are produced in the initial decryption phase. The verification avoids pairings entirely and only requires Multi-Scalar Multiplications (MSMs). In a full system, a helper can run the expensive decryption procedure to produce hints and broadcast them to the network. The verifier can then verify the decryption using only MSMs and hashes – thus trading bandwidth for compute.

In fact, our approach is quite general and is applicable to prior batched threshold encryption schemes but also to other pairing based advanced encryption schemes such as Timelock Encryption [BF01, DHMW23, GMR23], ABE [SW05], Distributed Broadcast Encryption [KMW23], Silent Threshold Encryption [GKPW24, GHK+25, GWWW25] and more.

Table 1 compares batched threshold encryption systems based on elliptic curves in which partial decryptions and ciphertexts are $O(1)$ group elements. All listed constructions have $O(B \log B)$ decryption complexity, except for the Batched IBE constructions [CGPW25, SAS24, AFP25, FPTX25], which have $O(B \log^2 B)$ decryption, and [GWWW25], which has $O(B^2)$ decryption. In [FPTX25], k denotes the number of batches

[2] On virtually all networks, the recommended requirements for validators is at most 32 vCPUs, which is driven by hardware availability/pricing to avoid imposing large operating burden on validators.

that can be decrypted before the public key has to be changed. For our indexed construction and [ABD+25], the table lists the constant-ciphertext-size instantiation; censorship resistance can be boosted by encrypting the same message to multiple indices, at the cost of a proportionally larger ciphertext (see Section 5).

Construction	Setup	$\lvert\mathsf{sk}\rvert$	$\lvert\mathsf{dk}\rvert$	$\lvert\mathsf{ct}\rvert$	E	CR	Assumption
Batch IBE [CGPW25], [SAS24], [AFP25]	DKG	$O(1)$	$O(B)$	$3[\mathbb{G}_1] + [\mathbb{G}_T]$	✗	B	GGM
BEAT-MEV [BFOQ25]	DKG	$O(1)$	$O(B)$	$3[\mathbb{G}_1] + [\mathbb{G}_T]$	✓	1	DBPDH
TrX [FPTX25]	DKG	$O(1)$	$O(kB)$	$2[\mathbb{G}_1] + [\mathbb{G}_T]$	✓	B	GGM
[GWWW25]	DKG	$O(1)$	$O(B)$	$2[\mathbb{G}_1] + [\mathbb{G}_T]$	✗	B	GGM
[ABD+25]	DKG	$O(1)$	$O(BN)$	$2[\mathbb{G}_1] + [\mathbb{G}_T]$	✓	1	B-DBPDH + AGM
[BNRT26]	MPC	$O(B)$	$O(BN)$	$2[\mathbb{G}_1] + [\mathbb{G}_T]$	✓	B	B-StatRankDef
[BNRT26][3]	MPC	$O(B)$	$O(BN)$	$[\mathbb{G}_1] + [\mathbb{G}_T]$	✓	B	B-QuadRankDef
This work, [ADG+26]	MPC	$O(B)$	$O(BN)$	$[\mathbb{G}_1] + [\mathbb{G}_T]$	✓	B	B-DBPDH
This work	MPC	$O(\delta)$	$O(B + \delta N)$	$2[\mathbb{G}_1] + [\mathbb{G}_T]$	✓	δ	(δ, B)-DBPDH

Table 1: Comparison of batched encryption systems based on elliptic curves where partial decryptions and ciphertexts are $O(1)$ group elements. $\lvert\mathsf{sk}\rvert$ denotes the per-party secret-key size, $\lvert\mathsf{dk}\rvert$ the total decryption-key size (including the CRS), and $\lvert\mathsf{ct}\rvert$ the ciphertext size with the NIZK proof omitted. Setup indicates distributed key generation (DKG) or multi-party computation (MPC). E indicates that the scheme does not suffer from epoch restrictions, and CR indicates censorship resistance. B denotes the maximum batch size, N the committee size, and $\delta \leq B$.

Evaluation. We implement the Simple BTE scheme in Rust using arkworks [ac22] and show that our scheme outperforms prior work (Section 8). At $B = 512$, we require just 2.80 seconds to decrypt the batch of ciphertexts. We also implement two different strategies for batch verification of decryption. At $B = 2048$:

- the bandwidth-optimized strategy is $17.6\times$ faster than computing the decryption locally and the hint is a PRG seed (16 bytes)
- the verification-optimized strategy is $114.1\times$ faster than computing the decryption locally but the hint is a target-group element (576 bytes on BLS12-381).

Concurrent Work. In concurrent work, [ADG+26] independently arrived at the same construction. After our paper and [ADG+26] appeared on the eprint archive, [BNRT26] posted a revised version which reduced the ciphertext size to $|\mathbb{G}_1| + |\mathbb{G}_T|$, matching our result but still using partial fraction techniques. We have updated our evaluation section to include a comparison against the implementation of [BNRT26].

2 Preliminaries

We use $[n]$ to denote the set $\{1, 2, \ldots, n\}$. The security parameter is denoted by $\lambda \in \mathbb{N}$. A function $f : \mathbb{N} \to \mathbb{N}$ is said to be polynomial if there exists a constant c such that $f(n) \leq n^c$ for all $n \in \mathbb{N}$, and we write $\mathsf{poly}(\cdot)$ to denote such a function. A function $f : \mathbb{N} \to [0, 1]$ is said to be negligible if for every $c \in \mathbb{N}$, there exists $N \in \mathbb{N}$ such that for all $n > N$, $f(n) < n^{-c}$, and we write $\mathsf{negl}(\cdot)$ to denote such a function. A probability is *noticeable* if it is not negligible, and *overwhelming* if it is equal to $1 - \mathsf{negl}(\lambda)$ for some negligible function $\mathsf{negl}(\lambda)$. For a set $\mathcal{S}$, we write $s \leftarrow \mathcal{S}$ to indicate that s is sampled uniformly at random from $\mathcal{S}$. An algorithm $\mathcal{A}$ is PPT (probabilistic polynomial-time) if its running time is bounded by some polynomial in the size of its input.

[3]The first version of the paper describes a construction with $O(B^2)$ decryption complexity, but in a revised version posted after a first version of our paper was released, the authors improved the decryption complexity to $O(B \log B)$ and reduced the ciphertext size to $[\mathbb{G}_1] + [\mathbb{G}_T]$.

Bilinear groups. Let $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, \circ) \leftarrow \mathsf{GrpGen}(1^\lambda)$ where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are groups of prime order p, g_1, g_2 are generators of $\mathbb{G}_1, \mathbb{G}_2$ respectively, and $\circ : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is an efficiently computable bilinear map satisfying (i) *bilinearity*: $[x]_1 \circ [y]_2 = [xy]_T$ for all $x, y \in \mathbb{F}_p$, and (ii) *non-degeneracy*: $g_1 \circ g_2 \neq 0_{\mathbb{G}_T}$. We write $g_T := g_1 \circ g_2$ and, for $b \in \{1, 2, T\}$, $[x]_b := x \cdot g_b$. We work in the Type-III (asymmetric) setting: no efficiently computable isomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$ is assumed.

Non-interactive Zero-Knowledge proofs. Let $\mathcal{L}$ be an NP-language and $\mathcal{R}$ the corresponding NP-relation. A Simulation Extractable-NIZK for $\mathcal{R}$ consists of the following algorithms:

- $\mathsf{Setup}(1^\lambda) \to (\mathsf{crs}, \mathsf{td})$: Takes as input a security parameter, and outputs a common reference string crs and trapdoor td.
- $\mathsf{Prove}(\mathsf{crs}, x, w) \to \pi$: Takes as input crs and any pair $(x, w) \in \mathcal{R}$ and outputs a proof π for the statement $x \in \mathcal{L}$.
- $\mathsf{Verify}(\mathsf{crs}, x, \pi) \to b$: Takes as input crs, statement x and proof π and outputs a bit b indicating whether verification has passed or failed.
- $\mathsf{SimProve}(\mathsf{crs}, \mathsf{td}, x) \to \pi$: Takes as input crs, trapdoor td and statement x and outputs a *simulated* proof π.

We will drop the crs term wherever implicit. When specifying a relation, we write $\{w \mid \varphi_1 \wedge \cdots \wedge \varphi_m\}$ where w is the witness and each φ_i is a constraint. At times we will have trivial constraints which just contain an unconstrained variable. In this case, it should be interpreted as a tag-based NIZK where the unconstrained variable is a part of the statement. A SE-NIZK satisfies the following properties:

- *Perfect Completeness.* An SE-NIZK satisfies perfect completeness if for any $(x, w) \in \mathcal{R}$, we have

$$\Pr\left[\begin{array}{c}(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda) \\ \pi \leftarrow \mathsf{Prove}(\mathsf{crs}, x, w)\end{array} : \mathsf{Verify}(x, \pi) = 1\right] = 1.$$

- *Proof of knowledge (and soundness).* For every PPT adversary $\mathcal{A}$, there is a PPT extractor Ext such that

$$\Pr\left[\begin{array}{c}(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}(\mathsf{crs}) \\ w \leftarrow \mathsf{Ext}(\mathsf{crs}, x, \pi)\end{array} : \begin{array}{c}\mathsf{Verify}(x, \pi) = 1 \\ (x, w) \notin \mathcal{R}\end{array}\right] \leq \mathsf{negl}(\lambda).$$

- *Computational Zero-knowledge.* There exists an algorithm SimProve such that for all PPT adversaries $\mathcal{A}$:

$$\left|\Pr\left[\begin{array}{c}(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda) \\ (x, w) \leftarrow \mathcal{A}(\mathsf{crs}) \\ \pi \leftarrow \mathsf{Prove}(\mathsf{crs}, x, w)\end{array} : \begin{array}{c}(x, w) \in \mathcal{R} \\ \mathcal{A}(\pi) = 1\end{array}\right] - \Pr\left[\begin{array}{c}(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda) \\ (x, w) \leftarrow \mathcal{A}(\mathsf{crs}) \\ \pi \leftarrow \mathsf{SimProve}(\mathsf{crs}, \mathsf{td}, x)\end{array} : \begin{array}{c}(x, w) \in \mathcal{R} \\ \mathcal{A}(\pi) = 1\end{array}\right]\right| \leq \mathsf{negl}(\lambda).$$

- *Simulation-Extractability.* For every PPT adversary $\mathcal{A}$, there exists a PPT extractor Ext such that

$$\Pr\left[\begin{array}{c}(\mathsf{crs}, \mathsf{td}) \leftarrow \mathsf{Setup}(1^\lambda) \\ (x, \pi) \leftarrow \mathcal{A}^{\mathsf{SimProve}(\mathsf{crs}, \mathsf{td}, \cdot)}(\mathsf{crs}) : \\ w \leftarrow \mathsf{Ext}(\mathsf{crs}, x, \pi)\end{array} \begin{array}{c}\mathsf{Verify}(x, \pi) = 1 \\ \wedge (x, w) \notin \mathcal{R} \\ \wedge (x, \pi) \notin Q\end{array}\right] \leq \mathsf{negl}(\lambda).$$

  where $\mathcal{A}$ has oracle access to $\mathsf{SimProve}(\mathsf{crs}, \mathsf{td}, \cdot)$, and Q is the list of statement-proof pairs returned by that oracle.

We will also demand that the proof is straight-line extractable. This means that the extractor can, on input a valid proof and (potentially) the list of random-oracle queries made by the adversary (prover), extract a witness with overwhelming probability without rewinding the prover. When the SE-NIZK is instantiated in the random-oracle model, the crs may be taken as empty and extraction uses the adversary's random-oracle queries.

3 Definitions

We recall the definitions of batched threshold encryption.

Definition 1 (Batched Threshold Encryption)**.** *A* batched threshold encryption (BTE) *scheme instantiated with $N \in \mathbb{N}$ parties, a threshold of $t \in [N]$, maximum batch size of $B \in \mathbb{N}$, and message space of $\mathcal{M}$ consists of the following algorithms:*

- $\mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t) \to (\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk})$*: The setup algorithm takes the security parameter λ, the maximum batch size B, the number of parties N, and the threshold t, and outputs a public encryption key* ek*, a public decryption key* dk*, and N secret keys* $(\mathsf{sk}_1, \ldots, \mathsf{sk}_N)$.
- $\mathsf{Enc}(\mathsf{ek}, m, \mathsf{idx}) \to \mathsf{ct}$*: The encryption algorithm takes the encryption key* ek*, a message $m \in \mathcal{M}$, and (optionally) an index* $\mathsf{idx} \in [B]$*, and outputs a ciphertext* ct.
- $\mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \mathsf{ct}_1, \mathsf{ct}_2, \ldots, \mathsf{ct}_B) \to \{0,1\}$*: The (deterministic) admissibility algorithm takes the public keys and B ciphertexts, and outputs 1 to indicate the batch of ciphertexts is decryptable and 0 otherwise.*
- $\mathsf{PartialDec}(\mathsf{ek}, \mathsf{sk}_j, \{\mathsf{ct}_i\}_{i\in[B]}) \to \mathsf{pd}_j$*: The (deterministic) partial-decryption algorithm takes the public encryption key* ek*, a secret key* sk_j *for a party $j \in [N]$, and B ciphertexts, and outputs a partial decryption share* pd_j *or $\perp$.*
- $\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i\in[B]}) \to \{0,1\}$*: The (deterministic) verification algorithm takes the decryption key* dk*, a party index $j \in [N]$, a claimed partial decryption share* pd_j*, and B ciphertexts, and outputs 1 iff* pd_j *is a valid share for party j on that batch.*
- $\mathsf{Combine}(\mathsf{dk}, T \subseteq [N], \{\mathsf{pd}_j\}_{j\in T}) \to \mathsf{pd}$*: The (deterministic) combiner algorithm takes the decryption key* dk*, a subset $T \subseteq [N]$, and the partial decryption shares of these $|T|$ parties, and outputs a partial decryption* pd *or $\perp$.*
- $\mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]}) \to \{m_i\}_{i\in[B]}$*: The (deterministic) decryption algorithm takes the decryption key* dk*, a partial decryption* pd *and B ciphertexts, and outputs the corresponding messages, where each $m_i \in \mathcal{M} \cup \{\perp\}$.*

Let $\mathsf{BTE} = (\mathsf{Setup}, \mathsf{Enc}, \mathsf{Admissible}, \mathsf{PartialDec}, \mathsf{Verify}, \mathsf{Combine}, \mathsf{Dec})$ be a BTE scheme. We call a batch $\{\mathsf{ct}_i\}_{i\in[B]}$ *admissible* with respect to $(\mathsf{ek}, \mathsf{dk})$ if $\mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i\in[B]}) = 1$.

Definition 2 (Correctness of BTE)**.** *We say* BTE *is* correct *if the following two properties hold for all $\lambda, B, N, t \in \mathbb{N}$ with $t \le N$.*

1. *For all $\{m_i\}_{i\in[B]} \in \mathcal{M}^B$, there exists an index assignment $\{\mathsf{idx}_i\}_{i\in[B]} \in [B]^B$ such that*

$$\Pr\left[\mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i\in[B]}) = 1 \;\middle|\; \begin{array}{l} (\mathsf{ek}, \{\mathsf{sk}_j\}_{j\in[N]}, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t), \\ \{\mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{ek}, m_i, \mathsf{idx}_i)\}_{i\in[B]} \end{array}\right] = 1.$$

2. *For all $T \subseteq [N]$ such that $|T| \ge t$, all $\{m_i\}_{i\in[B]} \in \mathcal{M}^B$, and all $\{\mathsf{idx}_i\}_{i\in[B]} \in [B]^B$,*

$$\Pr\left[\begin{array}{r} \mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i\in[B]}) = 0 \vee \\ \left(\begin{array}{r} \mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i\in[B]}) = 1 \;\forall j \in T \\ \wedge\, \mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]}) = (m_1, \ldots, m_B) \end{array}\right) \end{array} \;\middle|\; \begin{array}{l} (\mathsf{ek}, \{\mathsf{sk}_j\}_{j\in[N]}, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t), \\ \mathsf{ct}_i \leftarrow \mathsf{Enc}(\mathsf{ek}, m_i, \mathsf{idx}_i)\;\forall i \in [B], \\ \mathsf{pd}_j \leftarrow \mathsf{PartialDec}(\mathsf{ek}, \mathsf{sk}_j, \{\mathsf{ct}_i\}_{i\in[B]})\;\forall j \in T, \\ \mathsf{pd} \leftarrow \mathsf{Combine}(\mathsf{dk}, T, \{\mathsf{pd}_j\}_{j\in T}) \end{array}\right] = 1.$$

Definition 3 (B-IND-CCA Security of BTE)**.** BTE *is* B-IND-CCA-secure *if all non-uniform PPT adversaries $\mathcal{A}$ have negligible advantage in the following game:*

- *$\mathcal{A}$ first outputs a subset $S \subseteq [N]$ of size $t-1$ that it wishes to corrupt.*

- *The challenger runs* $(\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t)$ *and sends* ek*,* dk*, and* $\{\mathsf{sk}_j\}_{j \in S}$ *to* $\mathcal{A}$*.*
- ***Pre-challenge queries:*** *The adversary may issue an arbitrary number of queries for computing partial decryption shares:* $\mathcal{A}$ *sends a list* $\{\mathsf{ct}_i\}_{i \in [B]}$ *of* B *ciphertexts. If* $\mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i \in [B]}) = 0$*, the challenger returns* $\perp$ *for that query. Otherwise, the challenger computes* $\mathsf{pd}_j := \mathsf{PartialDec}(\mathsf{ek}, \mathsf{sk}_j, \{\mathsf{ct}_i\}_{i \in [B]})$ *for* $j \in [N] \setminus S$*, and sends them to* $\mathcal{A}$*.*
- ***Challenge round:*** $\mathcal{A}$ *sends two messages* $m_0, m_1 \in \mathcal{M}$ *and an index* $\mathsf{idx}^* \in [B]$*. The challenger samples* $b \leftarrow\!\$\ \{0,1\}$*, computes* $\mathsf{ct}^* \leftarrow \mathsf{Enc}(\mathsf{ek}, m_b, \mathsf{idx}^*)$*, and sends* ct^* *to* $\mathcal{A}$*.*
- ***Post-challenge queries:*** *The adversary can again issue an arbitrary number of queries for computing partial decryption shares. If* $\mathcal{A}$ *sends a list* $\{\mathsf{ct}_i\}_{i \in [B]}$ *such that* $\mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i \in [B]}) = 0$ *or* $\mathsf{ct}_i = \mathsf{ct}^*$ *for some* $i \in [B]$*, then the challenger returns* $\perp$ *for that query. Otherwise, the challenger computes* $\mathsf{pd}_j := \mathsf{PartialDec}(\mathsf{ek}, \mathsf{sk}_j, \{\mathsf{ct}_i\}_{i \in [B]})$ *for* $j \in [N] \setminus S$*, and sends them to* $\mathcal{A}$*.*
- ***Output:*** $\mathcal{A}$ *outputs a bit* $b' \in \{0,1\}$*.*

The adversary's advantage is

$$\mathsf{Adv}^{\mathsf{ind\text{-}cca}}_{\mathsf{BTE},\mathcal{A}}(\lambda, B, N, t) := \left|\Pr[b' = b] - \tfrac{1}{2}\right|.$$

Robustness. A BTE scheme must be robust against malicious decryption servers or clients that submit malformed ciphertexts or decryption queries. Informally, we require the following two properties:

- **Consistency:** It should be computationally infeasible for an adversary to produce two different sets of partial decryptions for the same admissible batch of ciphertexts such that both sets verify yet Combine yields conflicting plaintexts.
- **Rogue Ciphertext Security:** It should be computationally infeasible for an adversary to craft a malformed ciphertext that causes decryption of an honest party's ciphertext in the same admissible batch to fail.

Definition 4 (Rogue Ciphertext Security)**.** BTE *is* rogue ciphertext secure *if all non-uniform PPT adversaries* $\mathcal{A}$ *have a negligible probability of winning the following game:*

- *The challenger runs* $(\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t)$ *and sends everything to* $\mathcal{A}$*.*
- $\mathcal{A}$ *outputs a message* $m \in \mathcal{M}$ *and an index* $\mathsf{idx}^* \in [B]$*.*
- *The challenger computes* $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{ek}, m, \mathsf{idx}^*)$ *and sends* ct *to* $\mathcal{A}$*.*
- $\mathcal{A}$ *outputs* B *ciphertexts* $\{\mathsf{ct}_i\}_{i \in [B]}$*, a subset* $S \subseteq [N]$ *with* $|S| \geq t$*, and partial decryption shares* $\{\mathsf{pd}_j\}_{j \in S}$*.*
- *The adversary wins the game if:*
  - *there exists* $i^* \in [B]$ *such that* $\mathsf{ct} = \mathsf{ct}_{i^*}$*,*
  - $\mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i \in [B]}) = 1$*,*
  - $\mathsf{Verify}(\mathsf{dk}, \ell, \mathsf{pd}_\ell, \{\mathsf{ct}_i\}_{i \in [B]}) = 1$ *for all* $\ell \in S$*,*
  - $\mathsf{pd} \leftarrow \mathsf{Combine}(\mathsf{dk}, S, \{\mathsf{pd}_j\}_{j \in S})$ *succeeds and* $(m'_1, \ldots, m'_B) \leftarrow \mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i \in [B]})$*,*
  - $m'_{i^*} = \perp$ *or* $m'_{i^*} \neq m$*.*

Definition 5 (Consistency of BTE)**.** BTE *is* consistent *if all non-uniform PPT adversaries* $\mathcal{A}$ *have a negligible probability of winning the following game:*

- *The challenger runs* $(\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t)$ *and sends everything to* $\mathcal{A}$*.*

- *$\mathcal{A}$ outputs B ciphertexts $\{\mathsf{ct}_i\}_{i\in[B]}$, two subsets $S_1, S_2 \subseteq [N]$ with $|S_1|, |S_2| \geq t$, and partial decryptions $\{\mathsf{pd}_j^1\}_{j\in S_1}$ and $\{\mathsf{pd}_j^2\}_{j\in S_2}$.*
- *For each $b \in \{1,2\}$, the challenger computes $\mathsf{pd}^{(b)} \leftarrow \mathsf{Combine}(\mathsf{dk}, S_b, \{\mathsf{pd}_j^b\}_{j\in S_b})$. If $\mathsf{pd}^{(1)} = \perp$ or $\mathsf{pd}^{(2)} = \perp$, the adversary loses. Otherwise, it computes*

$$(m_1^{(b)}, \ldots, m_B^{(b)}) \leftarrow \mathsf{Dec}(\mathsf{dk}, \mathsf{pd}^{(b)}, \{\mathsf{ct}_i\}_{i\in[B]}).$$

- *The adversary wins if:*
  - $\mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i\in[B]}) = 1$,
  - $\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j^b, \{\mathsf{ct}_i\}_{i\in[B]}) = 1$ *for all* $b \in \{1,2\}$ *and all* $j \in S_b$,
  - $m_i^{(1)} \neq m_i^{(2)}$ *for some* $i \in [B]$.

4 Simple BTE

For simplicity, we first describe the CPA-secure core of the construction for a single server. We then explain how to add CCA security via a simulation-extractable NIZK and how to thresholdize the scheme. The full construction is described formally in Fig. 1.

- A trusted party runs the setup protocol and publishes the encryption key together with the punctured powers-of-τ values needed for decryption:

$$\mathsf{ek} := \left[\tau^{B+1}\right]_T \qquad \text{and} \qquad \mathsf{dk} := \left(\{h_j := \left[\tau^j\right]_2\}_{j\in[2B]\setminus\{B+1\}}\right).$$

- Encryption is then just ElGamal-style ciphertext in $\mathbb{G}_T$:

$$\mathsf{ct} := \left([k]_1,\ m + \left[k\tau^{B+1}\right]_T\right).$$

- To decrypt a batch $\{\mathsf{ct}_i\}_{i\in[B]}$, let $\mathsf{ct}_i = (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2})$ with $\mathsf{ct}_{i,1} = [k_i]_1$. It is sufficient to publish:

$$\mathsf{pd} := \sum_{i\in[B]} \tau^i \cdot \mathsf{ct}_{i,1} = \left[\sum_{i\in[B]} k_i\tau^i\right]_1.$$

- To decrypt the i-th ciphertext, one can compute:

$$\left[k_i\tau^{B+1}\right]_T = \mathsf{pd} \circ h_{B+1-i} - \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i}.$$

  Subtracting this mask from $\mathsf{ct}_{i,2}$ recovers m_i.

- To make the scheme CCA-secure, we augment each ciphertext with a simulation-extractable NIZK proof for the relation $\{k \mid \mathsf{ct}_1 = [k]_1 \wedge \mathsf{ct}_2\}$.
- To thresholdize this scheme, we secret share $\{\tau^i\}_{i\in[B]}$ among the N servers. Concretely, let server j hold shares $\{\sigma_j^i\}_{i\in[B]}$ of $\{\tau^i\}_{i\in[B]}$ and return the partial decryption

$$\mathsf{pd}_j := \sum_{i\in[B]} \sigma_j^i \cdot \mathsf{ct}_{i,1}.$$

  To verify these shares publicly, we additionally publish $\{v_j^i := \left[\sigma_j^i\right]_2\}_{(i,j)\in[B]\times[N]}$ as part of the trusted setup. Then anyone can check a claimed partial decryption by checking whether

$$\mathsf{pd}_j \circ g_2 = \sum_{i\in[B]} \mathsf{ct}_{i,1} \circ v_j^i.$$

Batched Threshold Encryption

$\underline{\mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t)}$:

- $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, \circ) \leftarrow \mathsf{GrpGen}(1^\lambda)$
- Sample $\tau \leftarrow \mathbb{F}_p$
- Run the setup for the chosen NIZK and let crs denote the resulting public parameters, if any.
- For each $i \in [B]$, secret-share τ^i among N servers with threshold t: server j receives share σ_j^i
- Output:

$$\mathsf{ek} := \left(\left[\tau^{B+1}\right]_T, \mathsf{crs}\right)$$
$$\mathsf{dk} := \left(\{h_j := \left[\tau^j\right]_2\}_{j\in[2B]\setminus\{B+1\}},\ \{v_j^i := \left[\sigma_j^i\right]_2\}_{(i,j)\in[B]\times[N]},\ \mathsf{crs}\right)$$
$$\mathsf{sk}_j := \left(\{\sigma_j^i\}_{i\in[B]}, \mathsf{crs}\right) \quad \text{for each server } j \in [N]$$

$\underline{\mathsf{Enc}(\mathsf{ek}, m, \mathsf{idx})}$: where $m \in \mathbb{G}_T$ and $\mathsf{idx} \in [B]$

- Parse $\mathsf{ek} = (E, \mathsf{crs})$
- Sample $k \leftarrow \mathbb{F}_p$ and set $\mathsf{ct}_1 := [k]_1$
- Set $\mathsf{ct}_2 := m + k \cdot E$
- Prove $\pi \leftarrow \left\{k \mid \mathsf{ct}_1 = [k]_1 \wedge \mathsf{ct}_2\right\}$
- Output $\mathsf{ct} := (\mathsf{ct}_1, \mathsf{ct}_2, \pi)$

$\underline{\mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i\in[B]})}$: Parse crs from dk and each $\mathsf{ct}_i = (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \pi_i)$. Output 1 iff $\mathsf{NIZK.Verify}(\mathsf{crs}, (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}), \pi_i) = 1$ for all $i \in [B]$.

$\underline{\mathsf{PartialDec}(\mathsf{ek}, \mathsf{sk}_j, \{\mathsf{ct}_i\}_{i\in[B]})}$:

- Parse $\mathsf{sk}_j = (\{\sigma_j^i\}_{i\in[B]}, \mathsf{crs})$ and $\{\mathsf{ct}_i = (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \pi_i)\}_{i\in[B]}$
- If $\mathsf{NIZK.Verify}(\mathsf{crs}, (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}), \pi_i) = 0$ for some $i \in [B]$, output $\perp$
- Output $\mathsf{pd}_j := \sum_{i\in[B]} \sigma_j^i \cdot \mathsf{ct}_{i,1}$

$\underline{\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i\in[B]})}$:

- Parse dk to recover $\{v_j^i\}_{(i,j)\in[B]\times[N]}$ and crs, and each $\mathsf{ct}_i = (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \pi_i)$.
- If $\mathsf{NIZK.Verify}(\mathsf{crs}, (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}), \pi_i) = 0$ for some $i \in [B]$, output 0. Otherwise, output 1 iff $\mathsf{pd}_j \circ g_2 = \sum_{i\in[B]} \mathsf{ct}_{i,1} \circ v_j^i$.

$\underline{\mathsf{Combine}(\mathsf{dk}, T, \{\mathsf{pd}_j\}_{j\in T})}$: Interpolate in the exponent to recover the pre decryption key: $\mathsf{pd} := \left[\sum_{i\in[B]} k_i \tau^i\right]_1$.

$\underline{\mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]})}$: For each $i \in [B]$, compute

$$m_i := \mathsf{ct}_{i,2} - \mathsf{pd} \circ h_{B+1-i} + \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i}$$

Figure 1: BTE scheme for N servers with threshold t, batch size B, and message space $\mathcal{M} = \mathbb{G}_T$.

Correctness. For Verify: an honest partial decryption satisfies $\mathsf{pd}_j = \sum_{i\in[B]} \sigma_j^i \cdot \mathsf{ct}_{i,1}$, so $\mathsf{pd}_j \circ g_2 = \sum_{i\in[B]} \mathsf{ct}_{i,1} \circ \left[\sigma_j^i\right]_2 = \sum_{i\in[B]} \mathsf{ct}_{i,1} \circ v_j^i$, which is exactly the check performed by Verify.

For Combine: since each $\sigma_j^i = f_i(j)$ for a degree-$(t-1)$ polynomial f_i with $f_i(0) = \tau^i$, Lagrange interpolation gives $\mathsf{pd} = \sum_{j\in T} \lambda_{T,j} \cdot \mathsf{pd}_j = \sum_{j\in T} \lambda_{T,j} \sum_{i\in[B]} f_i(j) \cdot \mathsf{ct}_{i,1} = \sum_{i\in[B]} \tau^i \cdot \mathsf{ct}_{i,1}$, which is the value used by Dec.

Since $\mathsf{ct}_{i,2} = \left[k_i \tau^{B+1}\right]_T + m_i$, the decryption algorithm computes:

$$
\begin{aligned}
m_i &= \mathsf{ct}_{i,2} - \mathsf{pd} \circ h_{B+1-i} + \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i} \\
&= \left[k_i \tau^{B+1}\right]_T + m_i - \left[\sum_{\ell\in[B]} k_\ell \tau^\ell\right]_1 \circ \left[\tau^{B+1-i}\right]_2 + \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \left[k_\ell\right]_1 \circ \left[\tau^{\ell+B+1-i}\right]_2 \\
&= m_i + \left[k_i \tau^{B+1}\right]_T - \sum_{\ell\in[B]} \left[k_\ell \tau^{\ell+B+1-i}\right]_T + \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \left[k_\ell \tau^{\ell+B+1-i}\right]_T \\
&= m_i + \left[k_i \tau^{B+1}\right]_T - \left[k_i \tau^{B+1}\right]_T = m_i,
\end{aligned}
$$

which proves correctness.

4.1 Security

Theorem 1 (Rogue ciphertext security)**.** *The construction in Fig. 1 satisfies rogue ciphertext security.*

Proof. Let $\mathcal{A}$ be an adversary in the game of Definition 4. Suppose it outputs a batch $\{\mathsf{ct}_i\}_{i\in[B]}$, a threshold set $S \subseteq [N]$, and shares $\{\mathsf{pd}_j\}_{j\in S}$ such that $\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i\in[B]}) = 1$ for every $j \in S$. Fix some $j \in S$. Since Verify accepts, every ciphertext proof is valid and

$$\mathsf{pd}_j \circ g_2 = \sum_{i\in[B]} \mathsf{ct}_{i,1} \circ v_j^i.$$

Because $\mathbb{G}_1$ is cyclic, for each $i \in [B]$ there exists $k_i \in \mathbb{F}_p$ such that $\mathsf{ct}_{i,1} = [k_i]_1$. Using $v_j^i = \left[\sigma_j^i\right]_2$, we get

$$\sum_{i\in[B]} \mathsf{ct}_{i,1} \circ v_j^i = \sum_{i\in[B]} [k_i]_1 \circ \left[\sigma_j^i\right]_2 = \sum_{i\in[B]} \left[k_i \sigma_j^i\right]_T = \Big(\sum_{i\in[B]} \sigma_j^i \cdot \mathsf{ct}_{i,1}\Big) \circ g_2.$$

By non-degeneracy of the pairing, this implies

$$\mathsf{pd}_j = \sum_{i\in[B]} \sigma_j^i \cdot \mathsf{ct}_{i,1},$$

so every accepted share is exactly the honest partial decryption share for server j on this batch.

Combining the threshold set therefore reconstructs

$$\mathsf{pd} = \left[\textstyle\sum_{\ell\in[B]} k_\ell \tau^\ell\right]_1.$$

Let i^* be the position containing the honest challenge ciphertext $\mathsf{ct} = \mathsf{Enc}(\mathsf{ek}, m, \mathsf{idx}^*)$, so that $\mathsf{ct}_{i^*,2} = m + \left[k_{i^*} \tau^{B+1}\right]_T$. Applying the Dec formula at coordinate i^*,

$$m'_{i^*} = \mathsf{ct}_{i^*,2} - \mathsf{pd} \circ h_{B+1-i^*} + \sum_{\substack{\ell\in[B]\\ \ell\neq i^*}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i^*} = \mathsf{ct}_{i^*,2} - \left[k_{i^*} \tau^{B+1}\right]_T = m,$$

where the middle equality uses only $\mathsf{ct}_{\ell,1} = [k_\ell]_1$: the terms with $\ell \neq i^*$ cancel, regardless of how the remaining ciphertexts were formed. Moreover, Dec always outputs an element of $\mathbb{G}_T$ (never $\perp$), so the case $m'_{i^*} = \perp$ is also excluded. Therefore the adversary can never win the game. □

Theorem 2 (Consistency)**.** *The construction in Fig. 1 is consistent.*

Proof. Let $\mathcal{A}$ be an adversary in the game of Definition 5. Suppose it outputs a batch $\{\mathsf{ct}_i\}_{i\in[B]}$, two subsets $S_1, S_2 \subseteq [N]$ of size at least t, and shares $\{\mathsf{pd}_j^1\}_{j\in S_1}$ and $\{\mathsf{pd}_j^2\}_{j\in S_2}$ that satisfy the winning conditions. Since Verify accepts every submitted share, all ciphertext proofs are valid and, for each $b \in \{1,2\}$ and each $j \in S_b$,

$$\mathsf{pd}_j^b \circ g_2 = \sum_{i\in[B]} \mathsf{ct}_{i,1} \circ v_j^i.$$

Because $\mathbb{G}_1$ is cyclic, for each $i \in [B]$ there exists a unique $k_i \in \mathbb{F}_p$ such that $\mathsf{ct}_{i,1} = [k_i]_1$. Using $v_j^i = [\sigma_j^i]_2$, the same calculation as in the proof of Theorem 1 shows that every accepted share must equal the honest partial decryption share:

$$\mathsf{pd}_j^b = \sum_{i\in[B]} \sigma_j^i \cdot \mathsf{ct}_{i,1}.$$

By uniqueness of polynomial interpolation, combining either threshold set recovers the same value:

$$\mathsf{pd}^{(1)} = \mathsf{Combine}(\mathsf{dk}, S_1, \{\mathsf{pd}_j^1\}_{j\in S_1}) = \mathsf{Combine}(\mathsf{dk}, S_2, \{\mathsf{pd}_j^2\}_{j\in S_2}) = \mathsf{pd}^{(2)}.$$

Since Dec is deterministic, decrypting the same ciphertext batch with the same reconstructed partial decryption yields the same message vector. Hence

$$\left\{m_i^{(1)} = m_i^{(2)}\right\}_{i\in[B]},$$

contradicting the adversary's winning condition. Therefore the adversary's winning probability is 0. □

To prove B-IND-CCA, we introduce the following hardness assumption.

Definition 6 (Decisional Bilinear q-Power Diffie-Hellman Assumption (q-DBPDH))**.** *Let* GrpGen *be an asymmetric bilinear group generator. The q-DBPDH problem is hard for* GrpGen *if the following two distributions are computationally indistinguishable:*

$$\mathcal{D}_0 = \left(\{[\tau^i]_1\}_{i\in[q]},\ \{[\tau^i]_2\}_{i\in[2q]\setminus\{q+1\}},\ [k]_1,\ [k\tau^{q+1}]_T\right),$$
$$\mathcal{D}_1 = \left(\{[\tau^i]_1\}_{i\in[q]},\ \{[\tau^i]_2\}_{i\in[2q]\setminus\{q+1\}},\ [k]_1,\ [z]_T\right),$$

where $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, \circ) \leftarrow \mathsf{GrpGen}(1^\lambda)$ *and* $\tau, k, z \leftarrow \mathbb{F}_p$.

The q-DBPDH assumption follows from the indexed (δ, q)-DBPDH assumption (Corollary 2), which we justify in the Type-III generic bilinear group model in Section A.

Remark 1. *Although we use an indistinguishability based assumption, one can use the weaker search variant by modifying the construction to derive a symmetric key from the $[k\tau^{B+1}]_T$ term via a random oracle and use that to encrypt the message.*

Theorem 3 (B-IND-CCA security)**.** *Assuming the B-DBPDH assumption holds and assuming the NIZK used in the construction is zero-knowledge and straight-line simulation-extractable, the construction in Fig. 1 is B-IND-CCA-secure.*

Remark 2 (On instantiating the NIZK)**.** *The CCA upgrade requires a proof of knowledge of a discrete logarithm. The most efficient instantiation is a Schnorr-like Σ-protocol made non-interactive via Fiat-Shamir. However, extraction for Schnorr relies on rewinding and does not yield straight-line simulation-extractability. Following prior work [CGPW25, AFP25, BFOQ25, FPTX25, ABD+25], we obtain straight-line extraction in the AGM [FKL18] / GGM [Sho97] at no additional cost. Alternatively, one can apply the Fischlin transform [Fis05] for straight-line extraction at the cost of a larger proof size.*

Proof. Let $\mathcal{A}$ be an adversary that wins the B-IND-CCA game (Definition 3) with non-negligible probability. Consider the following hybrids:

$\mathbf{H}_1$: This is the real game with challenge bit $b = 0$, so the challenge ciphertext is

$$\mathsf{ct}^* = \left([k^*]_1,\ m_0 + \left[k^*\tau^{B+1}\right]_T,\ \pi^*\right),$$

where π^* is generated honestly.

$\mathbf{H}_2$: This is identical to $\mathbf{H}_1$, except that the proof on the challenge ciphertext is generated with the NIZK simulation algorithm. The hybrids $\mathbf{H}_1$ and $\mathbf{H}_2$ are computationally indistinguishable by the zero-knowledge property of the NIZK. At this point, the challenger no longer needs the scalar k^*; it only uses the group elements $[k^*]_1$ and $\left[k^*\tau^{B+1}\right]_T$.

$\mathbf{H}_3$: This is identical to $\mathbf{H}_2$, except that the second ciphertext component is replaced by $m_0 + U$, where $U \leftarrow \mathbb{G}_T$ is uniform:

$$\mathsf{ct}^* = \left([k^*]_1,\ m_0 + U,\ \pi^*\right).$$

We show that $\mathbf{H}_2$ and $\mathbf{H}_3$ are computationally indistinguishable provided the B-DBPDH Assumption (Definition 6) holds. Suppose there exists an adversary distinguishing this hybrid from the previous one. We build a reduction $\mathcal{B}$ that wins the indistinguishability game for the B-DBPDH Assumption.

The reduction is given:

$$\left(\{g_i\}_{i\in[B]},\ \{h_j\}_{j\in[2B]\setminus\{B+1\}},\ [k]_1,\ Z\right),$$

and must decide whether $Z = \left[k\tau^{B+1}\right]_T$ or uniformly random in $\mathbb{G}_T$. At the beginning of the game, the reduction receives the corruption set $S \subseteq [N]$ of size $t-1$ from $\mathcal{A}$. It then computes $\left[\tau^{B+1}\right]_T := g_B \circ h_1$, runs the NIZK setup to obtain the crs and retains the NIZK trapdoor td (if applicable). It also sets

$$\mathsf{ek} := \left(\left[\tau^{B+1}\right]_T, \mathsf{crs}\right).$$

The public verification keys in dk are defined below.

For each slot $i \in [B]$ and each corrupted party $u \in S$, the reduction samples a random scalar share $\sigma_u^i \leftarrow \mathbb{F}_p$ and sets $\mathsf{sk}_u := (\{\sigma_u^i\}_{i\in[B]}, \mathsf{crs})$. For each honest party $j \notin S$ and each slot $i \in [B]$, let f_i denote the unique degree-$(t-1)$ polynomial with

$$f_i(0) = \tau^i \qquad \text{and} \qquad f_i(u) = \sigma_u^i \text{ for all } u \in S.$$

For corrupted parties, define

$$\widehat{\sigma}_u^i := \left[\sigma_u^i\right]_1 \qquad \text{and} \qquad v_u^i := \left[\sigma_u^i\right]_2.$$

Although $\mathcal{B}$ does not know $f_i(j)$ as a scalar for honest parties $j \notin S$, it can compute the corresponding group elements

$$\widehat{\sigma}_j^i := [f_i(j)]_1 \qquad \text{and} \qquad v_j^i := [f_i(j)]_2$$

by interpolation in the exponent from $g_i = \left[\tau^i\right]_1$, $h_i = \left[\tau^i\right]_2$, and the known values $\widehat{\sigma}_u^i = \left[\sigma_u^i\right]_1$ and $v_u^i = \left[\sigma_u^i\right]_2$ for $u \in S$. The reduction then sets

$$\mathsf{dk} := \left(\{h_j\}_{j\in[2B]\setminus\{B+1\}},\ \{v_j^i\}_{(i,j)\in[B]\times[N]},\ \mathsf{crs}\right).$$

It sends ek, dk, and $\{\mathsf{sk}_u\}_{u\in S}$ to $\mathcal{A}$.

To answer a PartialDec query on a batch $\{\mathsf{ct}_\ell\}_{\ell\in[B]}$, $\mathcal{B}$ applies the same rejection rules as in the B-IND-CCA game: in the post-challenge phase, if the query contains the exact challenge ciphertext ct^*, it returns $\perp$. Then $\mathcal{B}$ verifies all NIZK proofs; if any proof is invalid, it returns $\perp$, exactly as the real scheme does. Otherwise, for each ciphertext $\mathsf{ct}_\ell = (\mathsf{ct}_{\ell,1}, \mathsf{ct}_{\ell,2}, \pi_\ell)$, let $x_\ell = (\mathsf{ct}_{\ell,1}, \mathsf{ct}_{\ell,2})$ be the NIZK statement. If (x_ℓ, π_ℓ) is the simulated challenge statement-proof pair, then the query contains ct^* and was already rejected. Hence every accepting non-rejected proof is outside the simulator's proof list, and straight-line simulation-extractability yields, except with negligible probability, a witness k_ℓ such that $\mathsf{ct}_{\ell,1} = [k_\ell]_1$. For each honest party $j \notin S$, the reduction returns

$$\mathsf{pd}_j := \sum_{\ell\in[B]} k_\ell \cdot \widehat{\sigma}_j^\ell = \sum_{\ell\in[B]} [k_\ell f_\ell(j)]_1 \in \mathbb{G}_1,$$

which is exactly the honest partial decryption share.

When $\mathcal{A}$ outputs its challenge request, set $K := [k]_1$ and let $\mathcal{B}$ respond with

$$\mathsf{ct}^* := (K,\ m_0 + Z,\ \pi^*)$$

where $\pi^* \leftarrow \mathsf{SimProve}_{\mathsf{NIZK}}(\mathsf{crs}, \mathsf{td}, (K, m_0 + Z))$. We include $((K, m_0 + Z), \pi^*)$ in the list of simulated statement-proof pairs. That is, π^* is a simulated proof for the relation

$$\{k \mid K = [k]_1 \wedge m_0 + Z\}.$$

If $Z = \left[k\tau^{B+1}\right]_T$, then this is distributed exactly as the challenge ciphertext in $\mathbf{H}_2$; if Z is uniform in $\mathbb{G}_T$, then $m_0 + Z$ is also uniform in $\mathbb{G}_T$, so this is distributed exactly as $\mathbf{H}_3$. Therefore a distinguisher between $\mathbf{H}_2$ and $\mathbf{H}_3$ yields a distinguisher for the B-DBPDH assumption, leading to contradiction. Hence $\mathbf{H}_2$ and $\mathbf{H}_3$ are computationally indistinguishable.

Finally, $\mathbf{H}_3$ is message-independent: if U is uniform in $\mathbb{G}_T$, then both $m_0 + U$ and $m_1 + U$ are uniform in $\mathbb{G}_T$ as well. Hence replacing $m_0 + U$ by $m_1 + U$ does not change the distribution of the challenge ciphertext in $\mathbf{H}_3$. Starting from this identical message-independent hybrid, we can apply the same hybrid transitions in reverse with m_1 in place of m_0, obtaining the real game in which the challenge ciphertext encrypts m_1. Therefore the real games for challenge bits 0 and 1 are computationally indistinguishable, so the construction is B-IND-CCA-secure. □

5 Indexed Simple BTE

We define censorship resistance to be the *minimum* number of ciphertexts that need to be included in a batch before a *victim* ciphertext cannot be included in the batch. Ideally, this is the same as the maximum batch size B, i.e., to prevent a ciphertext from being included in the batch, an attacker must fill up the entire batch which translates to buying up the entire block. The construction in Fig. 1 provides maximum censorship resistance, however each party holds $O(B)$ secrets, so rotating committees requires either re-sharing $O(B)$ secrets or re-running the interactive setup.

In this section, we modify Fig. 1 to allow for a tradeoff between secret-key size and censorship resistance while paying a fixed ciphertext-size overhead. The resulting indexed scheme is parameterized by $\delta \in [B]$ and uses $D := \{-(\delta-1), \ldots, \delta-1\}$ such that a ciphertext $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{ek}, m, \mathsf{idx})$ can be placed in any batch position $i \in [B]$ satisfying $|i - \mathsf{idx}| < \delta$. Hence a ciphertext encrypted to index idx has

$$W(\mathsf{idx}) := \min\{B, \mathsf{idx} + \delta - 1\} - \max\{1, \mathsf{idx} - \delta + 1\} + 1$$

feasible positions, and to block such a ciphertext an attacker must fill all of them. The censorship resistance of the scheme is the minimum of this quantity over all positions, $\min_{\mathsf{idx}\in[B]} W(\mathsf{idx}) = \delta$.

- A trusted party samples $\tau, \mathsf{sk} \in \mathbb{F}_p^*$, publishes $\left[\tau^{B+1}\right]_T$ and the indexed encryption components $\{A_i := \left[\mathsf{sk}^{-1}\tau^i\right]_1\}_{i\in[B]}$, and keeps the usual punctured powers-of-τ values for decryption.
- Instead of sharing $\{\tau^i\}_{i\in[B]}$ as in Fig. 1, the setup only secret-shares $\{\mathsf{sk}\tau^d\}_{d\in D}$.
- To encrypt $m \in \mathbb{G}_T$ with index $\mathsf{idx} \in [B]$, the encryptor samples k and outputs

$$\mathsf{ct} = \left([k]_1,\ k \cdot A_{\mathsf{idx}},\ m + k \cdot \left[\tau^{B+1}\right]_T,\ \mathsf{idx}\right).$$

- Given a batch $\{\mathsf{ct}_\ell\}_{\ell\in[B]}$ with $\mathsf{ct}_\ell = (\mathsf{ct}_{\ell,1}, \mathsf{ct}_{\ell,2}, \mathsf{ct}_{\ell,3}, \mathsf{idx}_\ell)$, $\mathsf{Assign}_D(\mathsf{idx}_1, \ldots, \mathsf{idx}_B)$ outputs either $\perp$ or a permutation $\rho : [B] \to [B]$ such that $i - \mathsf{idx}_{\rho(i)} \in D$ for every $i \in [B]$; since $D = \{-(\delta-1), \ldots, \delta-1\}$, this is an interval assignment problem. Each ciphertext ℓ can be assigned to the interval $[a_\ell, b_\ell]$ where $a_\ell = \max\{1, \mathsf{idx}_\ell - \delta + 1\}$ and $b_\ell = \min\{B, \mathsf{idx}_\ell + \delta - 1\}$. We compute such a permutation using the unit-time scheduling algorithm for jobs with release times and deadlines [GJST81]; in our integer endpoint setting, this can be implemented in $O(B)$ time with buckets. If it succeeds, define the reindexed ciphertexts $\mathsf{ct}'_i := \mathsf{ct}_{\rho(i)}$ and the corresponding displacements $d_i := i - \mathsf{idx}_{\rho(i)}$. Server j then returns

$$\mathsf{pd}_j = \sum_{i\in[B]} \sigma_j^{d_i} \cdot \mathsf{ct}'_{i,2}.$$

- Combining t valid partial decryptions gives

$$\mathsf{pd} = \left[\sum_{i\in[B]} k_{\rho(i)}\tau^i\right]_1,$$

since $\sigma_j^{d_i} \cdot \mathsf{ct}'_{i,2}$ interpolates to $\mathsf{sk}\tau^{d_i} \cdot \left[k_{\rho(i)}\mathsf{sk}^{-1}\tau^{\mathsf{idx}_{\rho(i)}}\right]_1 = \left[k_{\rho(i)}\tau^i\right]_1$. The final decryption step is then the same pairing cancellation used in Fig. 1.

Remark 3. *Using a similar strategy as [ABD+25], we can also create κ independent ciphertexts for the same message, encrypted to indices $\mathsf{idx}_1, \ldots, \mathsf{idx}_\kappa$, at the cost of a κ-fold increase in ciphertext size. Let $F(\mathsf{idx}_\ell) := \{i \in [B] : |i - \mathsf{idx}_\ell| < \delta\}$ be the feasible window of the ℓ-th copy. Blocking all copies requires filling the union $\bigcup_{\ell\in[\kappa]} F(\mathsf{idx}_\ell)$, so the exact censorship resistance depends on the chosen indices. In particular, if the indices are spaced so that $|\mathsf{idx}_\ell - \mathsf{idx}_{\ell'}| \geq \delta$ for all $\ell \neq \ell'$, then each copy contributes at least δ fresh positions until the batch is full, providing censorship resistance of $\min\{\kappa\delta, B\}$. Our formal construction in Fig. 2 corresponds to $\kappa = 1$.*

The construction in Fig. 2 is a family of BTE schemes parameterized by the index radius $\delta \leq B$: for each fixed δ, it instantiates Definition 1, with Setup taking the additional parameter 1^δ. Like B, N, and t, the parameter δ is public and available to all algorithms.

Several algorithms must agree on a common assignment of ciphertexts to batch positions. We fix the following canonical rule: for $i = 1, \ldots, B$ in increasing order, assign to position i the yet-unassigned ciphertext with the smallest index idx among those satisfying $|i - \mathsf{idx}| < \delta$, breaking ties by position in the input batch. This earliest-deadline-first rule finds a valid assignment whenever one exists [GJST81]. All references to deterministically computing the permutation ρ in Fig. 2 refer to this rule.

5.1 Security

Correctness. For the indexed construction, set $D := \{-(\delta-1), \ldots, \delta-1\}$. The existence part of correctness follows by choosing $\mathsf{idx}_i = i$ for every $i \in [B]$. Then the identity permutation satisfies $i - \mathsf{idx}_i = 0 \in D$ for every $i \in [B]$, so $\mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i\in[B]}) = 1$ for every honestly generated batch with these indices.

Consider an honestly generated batch $\{\mathsf{ct}_\ell = (\mathsf{ct}_{\ell,1}, \mathsf{ct}_{\ell,2}, \mathsf{ct}_{\ell,3}, \mathsf{idx}_\ell, \pi_\ell)\}_{\ell\in[B]}$, where

$$\mathsf{ct}_{\ell,1} = [k_\ell]_1, \qquad \mathsf{ct}_{\ell,2} = \left[k_\ell \mathsf{sk}^{-1}\tau^{\mathsf{idx}_\ell}\right]_1, \qquad \mathsf{ct}_{\ell,3} = m_\ell + \left[k_\ell\tau^{B+1}\right]_T.$$

Indexed Batched Threshold Encryption

$\underline{\mathsf{Setup}(1^\lambda, 1^B, 1^\delta, 1^N, 1^t)}$:

- Run $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, \circ) \leftarrow \mathsf{GrpGen}(1^\lambda)$.
- Run the setup for the chosen NIZK and let crs denote the resulting public parameters.
- Set $D := \{-(\delta-1), \ldots, \delta-1\}$.
- Sample $\tau, \mathsf{sk}, \beta \leftarrow\!\$\ \mathbb{F}_p^*$.
- For each $d \in D$, secret-share $\mathsf{sk}\tau^d$ among N servers with threshold t; server j receives share σ_j^d.
- Output

$$\mathsf{ek} := \left(\left[\tau^{B+1}\right]_T,\ \left\{ A_i := \left[\mathsf{sk}^{-1}\tau^i\right]_1 \right\}_{i\in[B]}, \mathsf{crs} \right)$$

$$\mathsf{dk} := \left(\left\{ h_j := \left[\tau^j\right]_2 \right\}_{j\in[2B]\setminus\{B+1\}}, v_\beta := [\beta]_2,\ \left\{ v_j^d := \left[\beta\sigma_j^d\right]_2 \right\}_{(d,j)\in D\times[N]}, \{A_i\}_{i\in[B]},\ \mathsf{crs} \right)$$

$$\left\{ \mathsf{sk}_j := \left(\left\{\sigma_j^d\right\}_{d\in D}, \mathsf{crs} \right) \right\}_{j\in[N]}$$

$\underline{\mathsf{Enc}(\mathsf{ek}, m, \mathsf{idx})}$: where $m \in \mathbb{G}_T$ and $\mathsf{idx} \in [B]$

- Parse $\mathsf{ek} = (E, \{A_i\}_{i\in[B]}, \mathsf{crs})$.
- Sample $k \leftarrow\!\$\ \mathbb{F}_p$.
- Set $\mathsf{ct}_1 := [k]_1$, $\mathsf{ct}_2 := k \cdot A_{\mathsf{idx}}$, and $\mathsf{ct}_3 := m + k \cdot E$.
- Prove $\pi \leftarrow \mathsf{NIZK.Prove}(\mathsf{crs}, (\mathsf{ct}_1, \mathsf{ct}_2, \mathsf{ct}_3, \mathsf{idx}), \{k \mid \mathsf{ct}_1 = [k]_1 \wedge \mathsf{ct}_2 = k \cdot A_{\mathsf{idx}} \wedge \mathsf{ct}_3 \wedge \mathsf{idx}\})$.
- Output $\mathsf{ct} := (\mathsf{ct}_1, \mathsf{ct}_2, \mathsf{ct}_3, \mathsf{idx}, \pi)$.

$\underline{\mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i\in[B]})}$: Parse crs from dk and each $\mathsf{ct}_i = (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \mathsf{ct}_{i,3}, \mathsf{idx}_i, \pi_i)$. If $\mathsf{idx}_i \notin [B]$ for some $i \in [B]$, output 0. If $\mathsf{NIZK.Verify}(\mathsf{crs}, (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \mathsf{ct}_{i,3}, \mathsf{idx}_i), \pi_i) = 0$ for some $i \in [B]$, output 0. Using [GJST81] check whether there exists a permutation $\rho : [B] \to [B]$ such that $i - \mathsf{idx}_{\rho(i)} \in D$ for every $i \in [B]$; if so, output 1, otherwise output 0.

$\underline{\mathsf{PartialDec}(\mathsf{ek}, \mathsf{sk}_j, \{\mathsf{ct}_i\}_{i\in[B]})}$:

- Parse $\mathsf{ek} = (E, \{A_i\}_{i\in[B]}, \mathsf{crs})$, $\mathsf{sk}_j = (\{\sigma_j^d\}_{d\in D}, \mathsf{crs})$, and each $\mathsf{ct}_i = (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \mathsf{ct}_{i,3}, \mathsf{idx}_i, \pi_i)$.
- If $\mathsf{idx}_i \notin [B]$ for some $i \in [B]$, output $\perp$.
- If $\mathsf{NIZK.Verify}(\mathsf{crs}, (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \mathsf{ct}_{i,3}, \mathsf{idx}_i), \pi_i) = 0$ for some $i \in [B]$, output $\perp$.
- Using [GJST81], deterministically compute a permutation $\rho : [B] \to [B]$ such that $i - \mathsf{idx}_{\rho(i)} \in D$ for every $i \in [B]$, if one exists. If no such permutation exists, output $\perp$.
- Set $d_i := i - \mathsf{idx}_{\rho(i)}$.
- Output

$$\mathsf{pd}_j := \sum_{i\in[B]} \sigma_j^{d_i} \cdot \mathsf{ct}_{\rho(i),2}.$$

· · · continued on next page · · ·

· · · continued from previous page · · ·

$\underline{\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i\in[B]})}$:

- Parse dk to recover v_β, $\{v_j^d\}_{(d,j)\in D\times[N]}$, $\{A_i\}_{i\in[B]}$, and crs, and each $\mathsf{ct}_i = (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \mathsf{ct}_{i,3}, \mathsf{idx}_i, \pi_i)$.
- If $\mathsf{idx}_i \notin [B]$ for some $i \in [B]$, output 0.
- If $\mathsf{NIZK.Verify}(\mathsf{crs}, (\mathsf{ct}_{i,1}, \mathsf{ct}_{i,2}, \mathsf{ct}_{i,3}, \mathsf{idx}_i), \pi_i) = 0$ for some $i \in [B]$, output 0.
- Using [GJST81], deterministically compute $\rho : [B] \to [B]$ such that $i - \mathsf{idx}_{\rho(i)} \in D$ for every $i \in [B]$, if one exists. If no such permutation exists, output 0.
- Set $d_i := i - \mathsf{idx}_{\rho(i)}$. Output 1 iff

$$\mathsf{pd}_j \circ v_\beta = \sum_{i\in[B]} \mathsf{ct}_{\rho(i),2} \circ v_j^{d_i}.$$

$\underline{\mathsf{Combine}(\mathsf{dk}, T, \{\mathsf{pd}_j\}_{j\in T})}$: Interpolate in the exponent to recover the pre decryption key: $\mathsf{pd} := \left[\sum_{i\in[B]} k_{\rho(i)}\tau^i\right]_1$.

$\underline{\mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]})}$: If $\mathsf{idx}_i \notin [B]$ for some $i \in [B]$, output $(\perp, \ldots, \perp)$. Recompute the same permutation $\rho : [B] \to [B]$ using [GJST81]. If the permutation does not exist, output $(\perp, \ldots, \perp)$. Else, for $i \in [B]$, compute

$$m_{\rho(i)} := \mathsf{ct}_{\rho(i),3} - \mathsf{pd} \circ h_{B+1-i} + \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \mathsf{ct}_{\rho(\ell),1} \circ h_{\ell+B+1-i}.$$

Output $(m_1, \ldots, m_B)$.

Figure 2: BTE construction for N servers with threshold t, batch size B, index radius $\delta \leq B$, and $\mathcal{M} = \mathbb{G}_T$.

If the scheduling algorithm finds no permutation $\rho : [B] \to [B]$ satisfying $i - \mathsf{idx}_{\rho(i)} \in D$ for every $i \in [B]$, then Admissible outputs 0 (it runs the same feasibility check) and the correctness condition holds vacuously. Otherwise, fix the deterministically chosen permutation ρ, and write $\widehat{\mathsf{ct}}_i := \mathsf{ct}_{\rho(i)}$, $\widehat{\mathsf{idx}}_i := \mathsf{idx}_{\rho(i)}$, and $\widehat{k}_i := k_{\rho(i)}$ for the reindexed batch. Let $d_i := i - \widehat{\mathsf{idx}}_i \in D$.

For each $d \in D$, let f_d be the degree-$(t-1)$ sharing polynomial with $f_d(0) = \mathsf{sk}\tau^d$, so server j holds $\sigma_j^d = f_d(j)$. The partial decryption produced by server j is

$$\mathsf{pd}_j = \sum_{i\in[B]} \sigma_j^{d_i} \cdot \widehat{\mathsf{ct}}_{i,2} = \left[\sum_{i\in[B]} f_{d_i}(j)\widehat{k}_i \mathsf{sk}^{-1}\tau^{\widehat{\mathsf{idx}}_i}\right]_1 .$$

This share passes Verify, since by bilinearity

$$\mathsf{pd}_j \circ v_\beta = \sum_{i\in[B]} \widehat{\mathsf{ct}}_{i,2} \circ \left[\beta\sigma_j^{d_i}\right]_2 = \sum_{i\in[B]} \widehat{\mathsf{ct}}_{i,2} \circ v_j^{d_i}.$$

Combining any threshold set T gives

$$\begin{aligned}
\mathsf{pd} &= \sum_{j\in T} \lambda_{T,j} \cdot \mathsf{pd}_j \\
&= \left[\sum_{i\in[B]} \widehat{k}_i \mathsf{sk}^{-1}\tau^{\widehat{\mathsf{idx}}_i} \left(\sum_{j\in T} \lambda_{T,j} f_{d_i}(j)\right)\right]_1 \\
&= \left[\sum_{i\in[B]} \widehat{k}_i \mathsf{sk}^{-1}\tau^{\widehat{\mathsf{idx}}_i} \cdot \mathsf{sk}\tau^{d_i}\right]_1 = \left[\sum_{i\in[B]} \widehat{k}_i \tau^i\right]_1 ,
\end{aligned}$$

where the last equality uses $d_i = i - \widehat{\mathsf{idx}}_i$. Since $\widehat{\mathsf{ct}}_i = \mathsf{ct}_{\rho(i)}$, the direct formula in Dec computes, for each assigned position i,

$$\begin{aligned}
m_{\rho(i)} &= \widehat{\mathsf{ct}}_{i,3} - \mathsf{pd} \circ h_{B+1-i} + \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \widehat{\mathsf{ct}}_{\ell,1} \circ h_{\ell+B+1-i} \\
&= m_{\rho(i)} + \left[\widehat{k}_i\tau^{B+1}\right]_T - \left[\sum_{r\in[B]} \widehat{k}_r\tau^r\right]_1 \circ \left[\tau^{B+1-i}\right]_2 + \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \left[\widehat{k}_\ell\right]_1 \circ \left[\tau^{\ell+B+1-i}\right]_2 \\
&= m_{\rho(i)} + \left[\widehat{k}_i\tau^{B+1}\right]_T - \sum_{r\in[B]} \left[\widehat{k}_r\tau^{r+B+1-i}\right]_T + \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \left[\widehat{k}_\ell\tau^{\ell+B+1-i}\right]_T \\
&= m_{\rho(i)}.
\end{aligned}$$

Writing this value to output coordinate $\rho(i)$ for every $i \in [B]$ recovers the plaintexts in the original input-batch order.

Theorem 4 (Rogue ciphertext security)**.** *The construction in Fig. 2 satisfies rogue ciphertext security.*

Proof. Let $\mathcal{A}$ be an adversary in the game of Definition 4. Suppose it outputs an admissible batch $\{\mathsf{ct}_i\}_{i\in[B]}$, a threshold set $S \subseteq [N]$, and shares $\{\mathsf{pd}_j\}_{j\in S}$ such that $\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i\in[B]}) = 1$ for every $j \in S$. Let ρ be the deterministic assignment permutation and set $d_i := i - \mathsf{idx}_{\rho(i)}$.

Fix $j \in S$. Since Verify accepts, all NIZK proofs are valid and

$$\mathsf{pd}_j \circ v_\beta = \sum_{i\in[B]} \mathsf{ct}_{\rho(i),2} \circ v_j^{d_i}.$$

By soundness of the NIZK, for each ciphertext there exists k_i such that $\mathsf{ct}_{i,1} = [k_i]_1$ and $\mathsf{ct}_{i,2} = \left[k_i \mathsf{sk}^{-1} \tau^{\mathsf{idx}_i}\right]_1$. Using $v_\beta = [\beta]_2$ and $v_j^{d_i} = \left[\beta \sigma_j^{d_i}\right]_2$, we have

$$\sum_{i\in[B]} \mathsf{ct}_{\rho(i),2} \circ v_j^{d_i} = \left(\sum_{i\in[B]} \sigma_j^{d_i} \cdot \mathsf{ct}_{\rho(i),2}\right) \circ v_\beta.$$

Since $\beta \neq 0$, pairing with v_β is injective. Hence

$$\mathsf{pd}_j = \sum_{i\in[B]} \sigma_j^{d_i} \cdot \mathsf{ct}_{\rho(i),2},$$

and every accepted share is the honest partial decryption share for the assigned batch.

Combining the threshold set therefore reconstructs

$$\mathsf{pd} = \left[\sum_{i\in[B]} k_{\rho(i)} \tau^i\right]_1.$$

Let i^* be the input position containing the honest challenge ciphertext $\mathsf{ct} = \mathsf{Enc}(\mathsf{ek}, m, \mathsf{idx}^*)$, and let a^* be the assigned position with $\rho(a^*) = i^*$. Since the batch is admissible, such an a^* exists and $a^* - \mathsf{idx}^* \in D$. Applying the same cancellation calculation as in the correctness proof to the assigned position a^* gives $m'_{i^*} = m$. Thus $m'_{i^*} \neq \perp$, so the adversary wins only if soundness fails for some NIZK proof, which happens with negligible probability. □

Theorem 5 (Consistency)**.** *The construction in Fig. 2 satisfies consistency.*

Proof. Let $\mathcal{A}$ be an adversary in the game of Definition 5. Suppose it outputs an admissible batch $\{\mathsf{ct}_i\}_{i\in[B]}$, two threshold sets $S_1, S_2 \subseteq [N]$, and shares $\{\mathsf{pd}_j^1\}_{j\in S_1}$ and $\{\mathsf{pd}_j^2\}_{j\in S_2}$ satisfying the winning conditions. Let ρ be the deterministic assignment permutation and set $d_i := i - \mathsf{idx}_{\rho(i)}$.

Since Verify accepts every submitted share, the same calculation as in the proof of Theorem 4 shows that for every $b \in \{1, 2\}$ and every $j \in S_b$,

$$\mathsf{pd}_j^b = \sum_{i\in[B]} \sigma_j^{d_i} \cdot \mathsf{ct}_{\rho(i),2}.$$

By uniqueness of polynomial interpolation, combining either threshold set reconstructs the same aggregate partial decryption:

$$\mathsf{Combine}(\mathsf{dk}, S_1, \{\mathsf{pd}_j^1\}_{j\in S_1}) = \mathsf{Combine}(\mathsf{dk}, S_2, \{\mathsf{pd}_j^2\}_{j\in S_2}).$$

Since Dec is deterministic, decrypting the same admissible ciphertext batch with the same aggregate partial decryption yields the same message vector, contradicting the adversary's winning condition. Therefore the adversary's winning probability is 0. □

Definition 7 (Indexed (δ, q)-DBPDH)**.** *Let* GrpGen *be an asymmetric bilinear group generator, let $\delta \leq q$, and set $D := \{-(\delta-1), \ldots, \delta-1\}$. The indexed (δ, q)-DBPDH problem is hard for* GrpGen *if the following two distributions are computationally indistinguishable:*

$$\mathcal{D}_0 = \Big(\{[\tau^i]_1\}_{i\in[q]},\ \{[\alpha^{-1}\tau^i]_1\}_{i\in[q]},\ [k]_1,\ \{[k\alpha^{-1}\tau^i]_1\}_{i\in[q]},\ \{[\tau^i]_2\}_{i\in[2q]\setminus\{q+1\}}, [\beta]_2,\ \{[\beta\alpha\tau^i]_2\}_{i\in D},\ [k\tau^{q+1}]_T\Big),$$

$$\mathcal{D}_1 = \Big(\{[\tau^i]_1\}_{i\in[q]},\ \{[\alpha^{-1}\tau^i]_1\}_{i\in[q]},\ [k]_1,\ \{[k\alpha^{-1}\tau^i]_1\}_{i\in[q]},\ \{[\tau^i]_2\}_{i\in[2q]\setminus\{q+1\}}, [\beta]_2,\ \{[\beta\alpha\tau^i]_2\}_{i\in D},\ [z]_T\Big),$$

where $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, \circ) \leftarrow \mathsf{GrpGen}(1^\lambda)$, $\tau, \alpha, \beta \leftarrow\$\ \mathbb{F}_p^*$, *and* $k, z \leftarrow\$\ \mathbb{F}_p$.

We justify the indexed (δ, q)-DBPDH assumption in the Type-III generic bilinear group model in Section A.

Theorem 6 (B-IND-CCA security)**.** *Assuming the indexed (δ, B)-DBPDH assumption holds and assuming the NIZK used in the construction is zero-knowledge and straight-line simulation-extractable, the construction in Fig. 2 is B-IND-CCA-secure.*

Proof. Let $\mathcal{A}$ be a B-IND-CCA adversary. We prove indistinguishability of the games with challenge bits 0 and 1 using the following hybrids. The proof is identical to the proof of Theorem 3 except for the hybrid that replaces the challenge mask by a uniform element. In the present construction, the challenge ciphertext also contains the indexed component $\mathsf{ct}_2^* = \left[k\mathsf{sk}^{-1}\tau^{\mathsf{idx}^*}\right]_1$, so the reduction must be able to produce this term. This is exactly where we use indexed (δ, B)-DBPDH: its challenge includes $\{\left[k\alpha^{-1}\tau^i\right]_1\}_{i\in[B]}$, which becomes the required term after the reduction maps sk to α.

$\mathbf{H}_1$: This is the real B-IND-CCA game with challenge bit $b = 0$. The challenge ciphertext is

$$\mathsf{ct}^* = \left([k^*]_1\,,\ \left[k^*\mathsf{sk}^{-1}\tau^{\mathsf{idx}^*}\right]_1,\ m_0 + \left[k^*\tau^{B+1}\right]_T,\ \mathsf{idx}^*,\ \pi^*\right),$$

where π^* is generated honestly for the full ciphertext statement.

$\mathbf{H}_2$: This is identical to $\mathbf{H}_1$, except that π^* is generated using the NIZK simulator. The hybrids $\mathbf{H}_1$ and $\mathbf{H}_2$ are computationally indistinguishable by the zero-knowledge property of the NIZK.

$\mathbf{H}_3$: This is identical to $\mathbf{H}_2$, except that the mask in the third component is replaced by a uniform element $U \leftarrow \mathbb{G}_T$:

$$\mathsf{ct}^* = \left([k^*]_1\,,\ \left[k^*\mathsf{sk}^{-1}\tau^{\mathsf{idx}^*}\right]_1,\ m_0 + U,\ \mathsf{idx}^*,\ \pi^*\right).$$

We show that $\mathbf{H}_2$ and $\mathbf{H}_3$ are computationally indistinguishable under indexed (δ, B)-DBPDH.

Suppose there is a distinguisher between these hybrids. We build a reduction $\mathcal{B}$ that receives an indexed (δ, B)-DBPDH challenge

$$\left(\{\left[\tau^i\right]_1\}_{i\in[B]},\ \{\left[\alpha^{-1}\tau^i\right]_1\}_{i\in[B]},\ [k]_1\,,\ \{\left[k\alpha^{-1}\tau^i\right]_1\}_{i\in[B]},\ \{\left[\tau^i\right]_2\}_{i\in[2B]\setminus\{B+1\}},\ [\beta]_2\,,\ \{\left[\beta\alpha\tau^d\right]_2\}_{d\in D},\ Z\right),$$

and must decide whether $Z = \left[k\tau^{B+1}\right]_T$ or Z is uniform in $\mathbb{G}_T$.

At the beginning of the game, the reduction receives the corruption set $S \subseteq [N]$ of size $t-1$ from $\mathcal{A}$. It runs the NIZK setup to obtain crs and a simulation trapdoor, and interprets the scheme secret sk as α. It sets

$$E := \left[\tau^{B+1}\right]_T = \left[\tau^B\right]_1 \circ [\tau]_2\,, \qquad A_i := \left[\alpha^{-1}\tau^i\right]_1 \quad \text{for } i \in [B].$$

For each $d \in D$ and $u \in S$, $\mathcal{B}$ samples $\sigma_u^d \leftarrow\!\$\ \mathbb{F}_p$. It then interpolates in the exponent from $\left[\beta\alpha\tau^d\right]_2$ and the corrupted shares $\{\left[\beta\sigma_u^d\right]_2\}_{u\in S}$, where $\left[\beta\sigma_u^d\right]_2 := \sigma_u^d \cdot [\beta]_2$, to compute $v_j^d = \left[\beta\sigma_j^d\right]_2$ for every $(d, j) \in D \times [N]$, and publishes $\mathsf{ek} = (E, \{A_i\}_{i\in[B]}, \mathsf{crs})$ and

$$\mathsf{dk} = \left(\{h_j := \left[\tau^j\right]_2\}_{j\in[2B]\setminus\{B+1\}}, v_\beta := [\beta]_2\,,\ \{v_j^d\}_{(d,j)\in D\times[N]}, \{A_i\}_{i\in[B]},\ \mathsf{crs}\right).$$

Finally, it gives $\mathsf{sk}_u = (\{\sigma_u^d\}_{d\in D}, \mathsf{crs})$ to each corrupted party $u \in S$. These shares are distributed correctly, since fewer than t Shamir shares reveal no information about the constant term $\alpha\tau^d$.

To answer a pre-challenge or post-challenge partial-decryption query on a batch $\{\mathsf{ct}_\ell\}_{\ell\in[B]}$, $\mathcal{B}$ first applies the same rejection rules as the B-IND-CCA game: it returns $\perp$ if the batch is inadmissible, and in the post-challenge phase it also returns $\perp$ if the batch contains ct^*. Otherwise, $\mathcal{B}$ verifies all NIZK proofs. If any proof is invalid, it returns $\perp$. By straight-line simulation-extractability, for every accepted non-challenge ciphertext $\mathsf{ct}_\ell = (\mathsf{ct}_{\ell,1}, \mathsf{ct}_{\ell,2}, \mathsf{ct}_{\ell,3}, \mathsf{idx}_\ell, \pi_\ell)$, the reduction extracts k_ℓ such that $\mathsf{ct}_{\ell,1} = [k_\ell]_1$ and $\mathsf{ct}_{\ell,2} = \left[k_\ell\alpha^{-1}\tau^{\mathsf{idx}_\ell}\right]_1$. Let Q denote the total number of ciphertexts extracted across all decryption queries and $\varepsilon_{\mathsf{SE}}$ the simulation-extractability error of the NIZK. By a union bound, extraction fails for some accepted ciphertext with probability at most $Q \cdot \varepsilon_{\mathsf{SE}}$, which is negligible since Q is at most

B times the number of decryption queries. In the remainder of the argument we condition on all extractions succeeding, which changes the distinguishing advantage by at most $Q \cdot \varepsilon_{\mathsf{SE}}$.

Let ρ be the permutation and set $d_i := i - \mathsf{idx}_{\rho(i)}$. For an honest server $j \notin S$, let f_d denote the degree-$(t-1)$ sharing polynomial with $f_d(0) = \alpha\tau^d$ and $f_d(u) = \sigma_u^d$ for every $u \in S$. Let $\lambda_{0,j}$ and $\{\lambda_{u,j}\}_{u\in S}$ be the Lagrange coefficients such that

$$f_d(j) = \lambda_{0,j} \cdot f_d(0) + \sum_{u\in S} \lambda_{u,j} \cdot f_d(u).$$

Then $\mathcal{B}$ simulates the honest party's partial decryption as

$$\mathsf{pd}_j := \sum_{i\in[B]} \left(\lambda_{0,j} \cdot \left(k_{\rho(i)} \cdot \left[\tau^i\right]_1\right) + \sum_{u\in S} \lambda_{u,j} \cdot \left(\sigma_u^{d_i} \cdot \mathsf{ct}_{\rho(i),2}\right) \right).$$

This is exactly

$$\sum_{i\in[B]} f_{d_i}(j) \cdot \mathsf{ct}_{\rho(i),2},$$

because $f_{d_i}(u) = \sigma_u^{d_i}$ and

$$f_{d_i}(0) \cdot \mathsf{ct}_{\rho(i),2} = \alpha\tau^{d_i} \cdot \left[k_{\rho(i)}\alpha^{-1}\tau^{\mathsf{idx}_{\rho(i)}}\right]_1 = k_{\rho(i)} \cdot \left[\tau^i\right]_1,$$

where the last equality uses $d_i + \mathsf{idx}_{\rho(i)} = i$.

For the challenge ciphertext with index idx^*, $\mathcal{B}$ outputs

$$\mathsf{ct}^* = \left([k]_1,\ \left[k\alpha^{-1}\tau^{\mathsf{idx}^*}\right]_1,\ m_0 + Z,\ \mathsf{idx}^*,\ \pi^* \right),$$

where π^* is simulated. If $Z = \left[k\tau^{B+1}\right]_T$, this is distributed exactly as $\mathbf{H}_2$; if Z is uniform, then $m_0 + Z$ is uniform in $\mathbb{G}_T$, so this is distributed exactly as $\mathbf{H}_3$. Thus any distinguisher between $\mathbf{H}_2$ and $\mathbf{H}_3$ breaks indexed (δ, B)-DBPDH.

Finally, in $\mathbf{H}_3$ the third component is uniform in $\mathbb{G}_T$ and independent of the encrypted message. Therefore replacing $m_0 + U$ with $m_1 + U$ does not change the distribution. Starting from this identical message-independent hybrid, we reverse the same transitions with m_1 in place of m_0: first replace the uniform mask by $\left[k\tau^{B+1}\right]_T$ using indexed (δ, B)-DBPDH, and then replace the simulated challenge proof with an honest proof using zero-knowledge. This yields the real game with challenge bit $b = 1$. Hence the real challenge games for $b = 0$ and $b = 1$ are computationally indistinguishable. □

5.2 Admissibility under Random Indices

We now estimate how often a batch of ciphertexts encrypted to randomly chosen indices is admissible in the indexed construction. Consider a batch $\{\mathsf{ct}_\ell\}_{\ell\in[B]}$ of exactly B ciphertexts, and let $\mathcal{C} := \{\mathsf{ct}_\ell\}_{\ell\in[B]}$ denote the corresponding set of ciphertext labels, treating the labels as distinct even if two ciphertext values coincide. Each ciphertext ct_ℓ independently chooses an index $\mathsf{idx}_\ell \leftarrow\$\ [B]$ and can be assigned to any position in the interval

$$F_{\mathsf{ct}_\ell} := [\max\{1, \mathsf{idx}_\ell - \delta + 1\}, \min\{B, \mathsf{idx}_\ell + \delta - 1\}] \subseteq [B].$$

The batch is admissible exactly when there is a matching from ciphertexts to positions such that ct_ℓ is matched to a position in F_{ct_ℓ}.

For an interval of indices $I = \{a, a+1, \ldots, b\} \subseteq [B]$, define

$$S_I := \{\mathsf{ct}_\ell \in \mathcal{C} : \mathsf{idx}_\ell \in I\}$$

and let

$$I^{+\delta} := [\max\{1, a-\delta+1\}, \min\{B, b+\delta-1\}]$$

be the union of feasible positions for ciphertexts whose indices lie in I. We use the following form of Hall's theorem [Hal35]: for a finite family of sets $\{F_{\mathsf{ct}}\}_{\mathsf{ct}\in\mathcal{C}}$, there exists an injective assignment of each $\mathsf{ct} \in \mathcal{C}$ to an element of F_{ct} if and only if

$$|S| \le \left|\bigcup_{\mathsf{ct}\in S} F_{\mathsf{ct}}\right| \qquad \text{for every } S \subseteq \mathcal{C}.$$

Corollary 1. *The batch is admissible if and only if*

$$|S_I| \le |I^{+\delta}| \qquad \textit{for every interval } I \subseteq [B].$$

Proof. Suppose first that the batch is admissible. Fix an interval $I \subseteq [B]$. By Hall's theorem,

$$|S_I| \le \left|\bigcup_{\mathsf{ct}\in S_I} F_{\mathsf{ct}}\right|.$$

Since every ciphertext in S_I has index in I, we have $\bigcup_{\mathsf{ct}\in S_I} F_{\mathsf{ct}} \subseteq I^{+\delta}$. Hence $|S_I| \le |I^{+\delta}|$.

Conversely, suppose the interval condition holds but the batch is not admissible. By Hall's theorem, there exists a set $S \subseteq \mathcal{C}$ such that

$$|S| > \left|\bigcup_{\mathsf{ct}\in S} F_{\mathsf{ct}}\right|.$$

The union on the right may be disconnected, so decompose it into connected components $U_1, \dots, U_r$. For each component U_j, let $S_j \subseteq S$ be the ciphertexts whose feasible intervals lie in U_j. Since each F_{ct} is an interval, the sets $\{S_j\}_{j\in[r]}$ partition S. If $|S_j| \le |U_j|$ for every j, then

$$|S| = \sum_{j\in[r]} |S_j| \le \sum_{j\in[r]} |U_j| = \left|\bigcup_{\mathsf{ct}\in S} F_{\mathsf{ct}}\right|,$$

contradicting the choice of S. Thus at least one component U' is overloaded and let $S' \subseteq S$ denote the ciphertexts whose feasible intervals lie in U', then $|S'| > |U'|$. Let I' be the smallest interval containing the indices $\{\mathsf{idx}_\ell : \mathsf{ct}_\ell \in S'\}$. Since U' is connected, $(I')^{+\delta}$ is the union of feasible intervals for ciphertexts in S'. Therefore

$$|S_{I'}| \ge |S'| > |U'| = |(I')^{+\delta}|,$$

contradicting the interval condition. □

This characterization gives a direct probabilistic bound. The batch is inadmissible exactly when there exists an interval $I \subseteq [B]$ for which $|S_I| > |I^{+\delta}|$. There are $B(B+1)/2$ intervals in $[B]$, one for each choice of endpoints $1 \le a \le b \le B$. Fix such an interval $I = \{a, \dots, b\}$ and write $m_I := |I|$ and $s_I := |I^{+\delta}|$. Since each ciphertext independently chooses its index uniformly from $[B]$, the indicator of the event $\mathsf{idx}_\ell \in I$ is Bernoulli with probability m_I/B, and hence $|S_I|$ follows the binomial distribution

$$|S_I| \sim \mathrm{Bin}\Big(B, \frac{m_I}{B}\Big)$$

with B trials and success probability m_I/B. Thus, for this fixed interval, the probability that it violates the admissibility condition is

$$\Pr[|S_I| > s_I] = \Pr\Big[\mathrm{Bin}\Big(B, \frac{m_I}{B}\Big) > s_I\Big].$$

Taking a union bound over all possible interval violations gives

$$\Pr[\text{inadmissible}] \le \sum_{1\le a\le b\le B} \Pr\left[\mathrm{Bin}\left(B, \frac{b-a+1}{B}\right) > \min\{B, b+\delta-1\} - \max\{1, a-\delta+1\} + 1\right].$$

Equivalently, this can be written as the explicit binomial-tail bound

$$\Pr[\text{inadmissible}] \le \sum_{1\le a\le b\le B} \sum_{r=s_{a,b}+1}^{B} \binom{B}{r} \left(\frac{b-a+1}{B}\right)^r \left(1 - \frac{b-a+1}{B}\right)^{B-r},$$

where $s_{a,b} := \min\{B, b+\delta-1\} - \max\{1, a-\delta+1\} + 1$. We plot an upper bound on this probability (computed using the Chernoff bound below) for different batch sizes in Fig. 3.

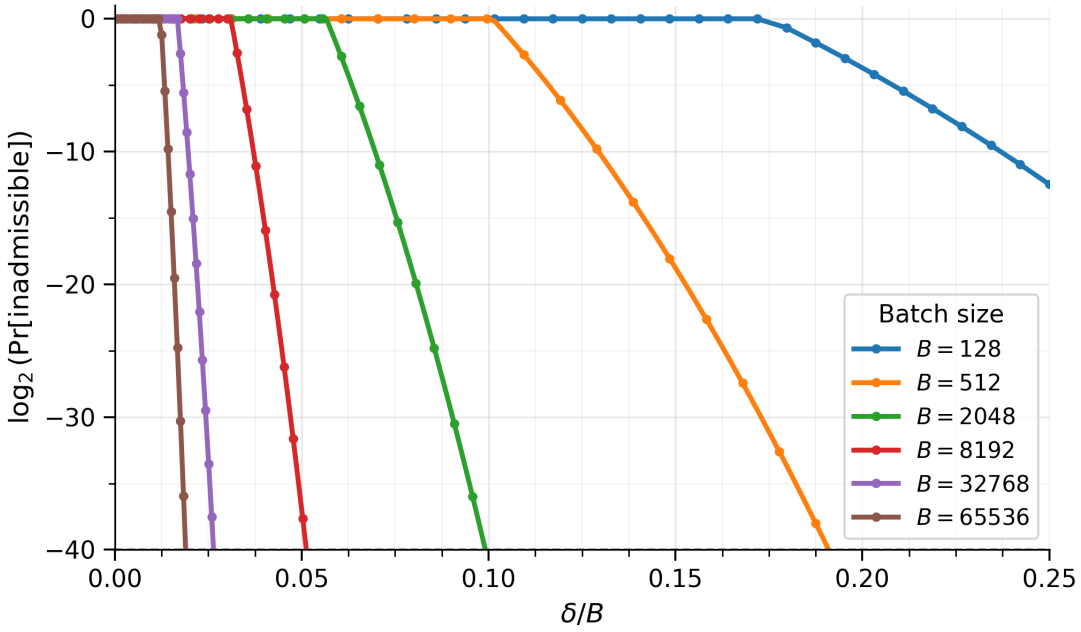

Figure 3: Upper bound on the probability that a randomly indexed batch is inadmissible for indexed BTE, as a function of the relative index parameter δ/B, for several batch sizes B. The probability decays rapidly with δ, especially at larger batch sizes.

Theorem 7 (Chernoff bound [Che52])**.** *Let $X \sim \mathrm{Bin}(n, p)$. For any $q > p$,*

$$\Pr[X \ge nq] \le \exp(-n \cdot D_{\mathrm{KL}}(q\|p)),$$

where

$$D_{\mathrm{KL}}(q\|p) := q \log \frac{q}{p} + (1-q) \log \frac{1-q}{1-p}$$

is the binary relative entropy.

For plotting, we use the following coarser approximation that groups intervals by their length. Intervals with $s_{a,b} = B$ can never be violated, so we drop them and apply the Chernoff bound to each remaining term in the interval union bound:

$$\Pr[\text{inadmissible}] \le \sum_{\substack{1\le a\le b\le B \\ s_{a,b}<B}} \exp\left(-B \cdot D_{\mathrm{KL}}\left(\frac{s_{a,b}+1}{B} \middle\| \frac{b-a+1}{B}\right)\right).$$

Now fix an interval length $m = b - a + 1$. There are $B - m + 1$ intervals of this length, and for each of them

$$s_{a,b} \geq s_m := m + \min\{\delta - 1, B - m\}.$$

Since $D_{\mathrm{KL}}(q\|p)$ is increasing for $q > p$, replacing $s_{a,b}$ by the smaller quantity s_m can only increase the Chernoff term and gives the upper bound

$$\Pr[\text{inadmissible}] \leq \sum_{\substack{m\in[B]\\ s_m<B}} (B - m + 1) \cdot \exp\left(-B \cdot D_{\mathrm{KL}}\left(\frac{s_m + 1}{B}\middle\|\frac{m}{B}\right)\right).$$

The right-hand sides above are nonincreasing in δ, since increasing δ only increases the thresholds $s_{a,b}$ and s_m.

6 Optimizing Decryption

We now describe two optimizations to the decryption algorithm. The first is a FFT-style acceleration of the decryption procedure that reduces the complexity from $O(B^2)$ to $O(B \log B)$. The second is a pipelining optimization that outlines how we can pipeline the FFT decryption work even before the partial decryptions are available such that the *final* step only requires $O(B)$ pairings.

6.1 FFT Acceleration

The decryption procedure in Fig. 1 requires $O(B^2)$ pairings to decrypt a batch of size B if implemented directly. We can reduce this to $O(B)$ pairings and $O(B \log B)$ group operations by exploiting the Toeplitz structure of the masking terms via FFT-style convolution. The expensive part of decryption is computing the family of cross-terms

$$C_i := \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i} \qquad \text{for } i \in [B].$$

This is a Toeplitz correlation: the $\mathbb{G}_2$ index depends only on the difference $\ell - i$. That structure lets us compute all C_i simultaneously by an FFT-style evaluation/interpolation procedure, rather than recomputing each sum independently.

For simplicity, assume B is a power of two and that $\mathbb{F}_p$ contains a primitive $2B$-th root of unity ω. We use transform length $L = 2B$. Define the $\mathbb{G}_1$-valued sequence

$$a_i := \begin{cases} \mathsf{ct}_{i,1} & \text{for } i \in [B], \\ 0 & \text{otherwise,} \end{cases}$$

and the $\mathbb{G}_2$-valued sequence indexed by shifts $d \in \{-(B-1), \ldots, B-1\}$,

$$b_d := \begin{cases} h_{B+1+d} & \text{if } d \neq 0, \\ 0 & \text{if } d = 0, \end{cases}$$

again padded by zeros to length $2B$. Then

$$C_i = \sum_{\ell\in[B]} a_\ell \circ b_{\ell-i},$$

where the $d = 0$ term vanishes because we set $b_0 = 0$. Now take the DFTs of these sequences, with coefficients in the source groups:

$$\widehat{A}_r := \sum_{i=0}^{L-1} \omega^{ri} a_i \in \mathbb{G}_1, \qquad \widehat{B}_r := \sum_{d=0}^{L-1} \omega^{-rd} b_d \in \mathbb{G}_2,$$

for $r \in [0, 2B-1]$. By bilinearity,

$$\widehat{C}_r := \widehat{A}_r \circ \widehat{B}_r = \sum_{i,d} \omega^{r(i-d)} \big(a_i \circ b_d\big) \in \mathbb{G}_T.$$

Applying the inverse DFT in $\mathbb{G}_T$ recovers

$$C_i = \frac{1}{2B} \sum_{r=0}^{2B-1} \omega^{-ri} \widehat{C}_r \qquad \text{for each } i \in [B].$$

So all cross-terms can be computed from:

- one FFT over $\mathbb{G}_1$ to form the $\widehat{A}_r$ values,
- one FFT over $\mathbb{G}_2$ to form the $\widehat{B}_r$ values,
- $2B = O(B)$ pairings to compute the $\widehat{C}_r$ values,
- one inverse FFT over $\mathbb{G}_T$,
- B pairings to compute $\mathsf{pd} \circ h_{B+1-i}$ for each $i \in [B]$.

The total cost is $O(B)$ pairings and $O(B \log B)$ group operations. Once the C_i are available, one can recover the messages as

$$m_i = \mathsf{ct}_{i,2} - \mathsf{pd} \circ h_{B+1-i} + C_i$$

for all $i \in [B]$.

6.2 Pipelined Decryption

We observe that the cross-term computation:

$$C_i = \sum_{\substack{\ell \in [B] \\ \ell \neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i}$$

depends only on the ciphertexts $\{\mathsf{ct}_{i,1}\}_{i \in [B]}$ and the public parameters $\{h_j\}$ in dk. In particular, it does *not* depend on the combined partial decryption pd, which is only available after PartialDec and Combine complete.

This means the bulk of the FFT work — the forward DFTs over $\mathbb{G}_1$ and $\mathbb{G}_2$, the $2B$ frequency-domain pairings, and the inverse DFT over $\mathbb{G}_T$ — can all be *pipelined* as soon as the batch of ciphertexts is known (e.g., once a block is proposed), without waiting for any partial decryption shares.

Concretely, decryption splits into two phases:

1. **Pre-decryption phase**:
   - Compute $\widehat{A}_r$ via a $\mathbb{G}_1$-FFT
   - Compute $\widehat{B}_r$ via a $\mathbb{G}_2$-FFT
   - Compute $\widehat{C}_r := \widehat{A}_r \circ \widehat{B}_r$ for each $r \in [0, 2B-1]$ ($2B$ pairings)
   - Recover all C_i via an inverse $\mathbb{G}_T$-FFT
2. **Finalization phase**:
   - Compute $\mathsf{pd} \circ h_{B+1-i}$ for each $i \in [B]$ (B pairings)
   - Recover $m_i = \mathsf{ct}_{i,2} - \mathsf{pd} \circ h_{B+1-i} + C_i$ for each $i \in [B]$

The pre-decryption phase accounts for $2B$ pairings and $O(B \log B)$ group operations (three FFTs), while the finalization phase requires only B pairings and B group operations in $\mathbb{G}_T$. In a system where PartialDec, Verify, and Combine introduce latency (e.g., due to network round-trips in a distributed validator set), the pre-decryption phase can be pipelined with these steps, resulting in the overall additional latency dominated by just $O(B)$ pairings + group operations.

7 Batch Verification of Decryption

[GHK+25] showed that many pairing-based *advanced* encryption schemes, such as IBE/Timelock Encryption [BF01, DHMW23, GMR23], ABE [SW05], Batch Threshold Encryption/IBE [CGPW25, AFP25], Distributed Broadcast Encryption [KMW23], and Silent Threshold Encryption [GKPW24], can be viewed as witness encryptions for relations of the form:

$$\underbrace{\begin{bmatrix} A_{1,1} & \cdots & A_{1,n} \\ A_{2,1} & \cdots & A_{2,n} \\ \vdots & \ddots & \vdots \\ A_{m,1} & \cdots & A_{m,n} \end{bmatrix}}_{\text{statement}} \circ \underbrace{\begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}}_{\text{witness}} = \underbrace{\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}}_{\text{statement}},$$

where the entries of A and w live in compatible source groups, and the entries of b live in $\mathbb{G}_T$. For such a relation, encryption samples $\alpha \leftarrow \mathbb{F}_p^m$ and computes $\mathsf{Enc}(\mathsf{x} = (A, b), M)$:

$$\mathsf{ct}_0 := M + \sum_{i \in [m]} \alpha_i \cdot b_i, \qquad \mathsf{ct}_j := \sum_{i \in [m]} \alpha_i \cdot A_{i,j} \quad \text{for } j \in [n].$$

Given a valid witness w, decryption recovers the message as $\mathsf{Dec}(\mathsf{w}, \mathsf{ct})$:

$$\mathsf{Dec}(\mathsf{w}, \mathsf{ct}) : M \leftarrow \mathsf{ct}_0 - \sum_{j \in [n]} \mathsf{ct}_j \circ w_j \tag{1}$$

First Attempt. In some pairing-based proof systems, it is possible to reduce the verification of N pairing-product equations to MSMs and a single pairing-product equation [FGHP09]. A natural approach is to have the helper send the recovered message together with the corresponding witness as the hint. The verifier can then check that the decryption equation Eq. (1) is satisfied for all of the ciphertexts with a random linear-combination test. However, the number of pairings only reduces when there is a common input across the different pairing terms $\prod_{i=1}^{B} e(g_i, h)^{r_i} = e\left(\sum_{i=1}^{B} r_i g_i,\ h\right)$.

In the context of witness encryption, if all ciphertexts use the same witness, then the verifier can fold pairings across the B ciphertexts with an MSM. Fortunately, this is the case for timelock encryption, where the witness is a signature on the block height. However, if each ciphertext uses a different witness, as is the case in most other applications of pairing-based WE schemes, the verifier will have to evaluate a large number of pairings.

Our Approach. We start from the following observation:

A perfectly correct encryption scheme is also a perfectly binding commitment scheme.

As a consequence, decryption can be verified by simply *re-encrypting* the ciphertext using the claimed message and randomness. Moreover, encryption (and therefore verification of decryption) in the above WE scheme does not involve pairings and can be batched across ciphertexts even when encrypting to different statements.

For a batch of B ciphertexts, let ct^k denote the k-th ciphertext and let (M^k, α^k) be the helper's claimed opening. Rather than rechecking each ciphertext independently, the verifier samples $r \leftarrow \mathbb{F}_p^B$ and checks the random linear combination

$$\sum_{k \in [B]} r_k \cdot \mathsf{ct}_j^k \stackrel{?}{=} \sum_{k \in [B]} \sum_{i \in [m]} r_k \alpha_i^k \cdot A_{i,j}^k \qquad \text{for each } j \in [n],$$

and

$$\sum_{k \in [B]} r_k \cdot (\mathsf{ct}_0^k - M^k) \stackrel{?}{=} \sum_{k \in [B]} \sum_{i \in [m]} r_k \alpha_i^k \cdot b_i^k.$$

These are MSM checks in the source groups and $\mathbb{G}_T$; no pairings are needed. The soundness follows from the standard random-linear-combination argument used for batch verification [FGHP09, Theorem 3.2]. One may also use small ℓ-bit coefficients, in which case an invalid batch is accepted with probability at most about $2^{-\ell}$.

It remains to show how to recover the randomness from the ciphertext. This can be achieved generically by encrypting the randomness under a separate ciphertext, but we describe a more efficient approach below.

Randomness Recovery. We use the Fujisaki-Okamoto transform [FO98, FO13] to achieve randomness recovery. Note that this does not provide CCA security in the threshold setting, as validators would have to perform certain checks securely inside MPC before releasing partial decryptions, which would require multiple rounds of interaction. Instead, we opt to use a proof of knowledge as in Section 4 for CCA security and only use the FO transform for randomness recovery. Let $\mathcal{K}$ be the message space of the witness-encryption scheme, and

$$H_R : \mathcal{K} \times \{0,1\}^{\ell_m} \to \{0,1\}^{\ell_\rho}, \qquad H_M : \mathcal{K} \to \{0,1\}^{\ell_m}$$

be random oracles. Let $G : \{0,1\}^{\ell_\rho} \to \mathbb{F}_p^m$ be a PRG that expands a short seed into the randomness vector used by the linear scheme. The transformed witness encryption scheme is:

- $\mathsf{Enc}'(\mathsf{x}, M)$ samples $K \leftarrow \mathcal{K}$, derives $\rho := H_R(K, M)$ and $\alpha := G(\rho)$, and outputs

$$\mathsf{ct} := (\mathsf{Enc}(\mathsf{x}, K; \alpha),\ H_M(K) \oplus M)\,.$$

- $\mathsf{Dec}'(\mathsf{w}, \mathsf{ct})$ parses $\mathsf{ct} = (\mathsf{ct}_1, \mathsf{ct}_2)$, computes $K \leftarrow \mathsf{Dec}(\mathsf{w}, \mathsf{ct}_1)$, derives $M := \mathsf{ct}_2 \oplus H_M(K)$, $\rho := H_R(K, M)$, and $\alpha := G(\rho)$, and accepts only if $\mathsf{Enc}(\mathsf{x}, K; \alpha) = \mathsf{ct}_1$; otherwise it outputs $\perp$.

We now give two possible versions of the hint format for a batch of ciphertexts $\{\mathsf{ct}^k = (\widehat{\mathsf{ct}}^{\,k}, \mathsf{ct}_2^k)\}_{k\in[B]}$, where $\widehat{\mathsf{ct}}^{\,k} = (\widehat{\mathsf{ct}}_0^k, \widehat{\mathsf{ct}}_1^k, \ldots, \widehat{\mathsf{ct}}_n^k)$ denotes the underlying witness-encryption ciphertext.

Bandwidth-optimized hints. The helper sends the short PRG seeds (16 bytes each) $\{\rho^k\}_{k\in[B]}$ and the verifier expands $\alpha^k := G(\rho^k)$. For each $k \in [B]$, the verifier computes:

$$K^k := \widehat{\mathsf{ct}}_0^k - \sum_{i\in[m]} \alpha_i^k \cdot b_i^k, \quad M^k := \mathsf{ct}_2^k \oplus H_M(K^k), \tag{2}$$

which requires B target-group MSMs of size m. The verifier then checks, for all $k \in [B]$, that $\rho^k = H_R(K^k, M^k)$ and that $\widehat{\mathsf{ct}}_j^k = \sum_{i\in[m]} \alpha_i^k \cdot A_{i,j}^k$ for all $j \in [n]$. The latter can be batch-verified using standard techniques [FGHP09].

Verification-optimized hints. The helper sends $\{K^k\}_{k\in[B]}$ directly, each of which is 576 bytes on BLS12-381. The verifier computes $M^k := \mathsf{ct}_2^k \oplus H_M(K^k)$ and $\rho^k := H_R(K^k, M^k)$, expands $\alpha^k := G(\rho^k)$, and checks that $\widehat{\mathsf{ct}}_j^k = \sum_{i\in[m]} \alpha_i^k \cdot A_{i,j}^k$ for all $j \in [n]$. Now, instead of B target-group MSMs, the verifier additionally samples $r \leftarrow \mathbb{F}_p^B$ and checks that

$$\sum_{k\in[B]} r_k \cdot (\widehat{\mathsf{ct}}_0^k - K^k) \stackrel{?}{=} \sum_{k\in[B]} \sum_{i\in[m]} r_k \alpha_i^k \cdot b_i^k,$$

which only requires target-group MSMs.

Malformed ciphertexts. The fast path above assumes the helper can recover the randomness for all ciphertexts. However, a malicious user can always submit a malformed ciphertext that fails the decryption checks. In this case, the helper cannot simply output $\perp$, because a malicious helper could suppress a valid ciphertext by falsely claiming it is malformed.

One strategy is to have the helper provide the witness for the underlying relation as part of the hint. This allows the verifier to run the decryption locally and confirm that the ciphertext is malformed. Indeed, this

would work for [CGPW25, AFP25], since the witness is $O(1)$ group elements. But as we will see in Section 7.2, the witness for the Simple BTE relation is actually $O(B)$ group elements and decryption requires $O(B)$ pairings. Naively using the above strategy would quickly destroy the benefit of helper-aided verification. In Section 7.4, we show how to certify malformed ciphertexts more efficiently.

7.1 Definitions

A *helper-aided decryption scheme* for a BTE scheme $(\mathsf{Setup}, \mathsf{Enc}, \mathsf{Admissible}, \mathsf{PartialDec}, \mathsf{Verify}, \mathsf{Combine}, \mathsf{Dec})$ consists of two algorithms:

- $\mathsf{HintGen}(\mathsf{ek}, \mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]}) \to \mathsf{hint}$: takes the public encryption and decryption keys, the combined partial decryption, and the batch, and outputs a hint.
- $\mathsf{HintDec}(\mathsf{ek}, \mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]}, \mathsf{hint}) \to \{m_i\}_{i\in[B]}$ or $\perp$: the (deterministic) hint-decryption algorithm verifies the hint and either outputs plaintexts $m_i \in \mathcal{M} \cup \{\perp\}$ for every slot, or rejects the hint by outputting $\perp$.

For completeness, we demand that an honestly generated hint makes HintDec agree *exactly* with the (deterministic) decryption algorithm Dec on the same inputs. Importantly, this must hold even when the ciphertexts or partial decryptions are generated maliciously.

Definition 8 (Completeness)**.** *A helper-aided decryption scheme is* complete *if the following holds for all $\lambda, B, N, t \in \mathbb{N}$ with $t \le N$. For every key tuple,*

$$(\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t),$$

every admissible batch of ciphertexts,

$$\{\mathsf{ct}_i\}_{i\in[B]} \quad \textit{such that} \quad \mathsf{Admissible}(\mathsf{ek}, \mathsf{dk}, \{\mathsf{ct}_i\}_{i\in[B]}) = 1,$$

and every subset $T \subseteq [N]$ with $|T| \ge t$ together with partial decryptions that verify,

$$\{\mathsf{pd}_j\}_{j\in T} \quad \textit{such that} \quad \mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i\in[B]}) = 1 \ \ \forall j \in T,$$

if $\mathsf{pd} := \mathsf{Combine}(\mathsf{dk}, T, \{\mathsf{pd}_j\}_{j\in T}) \neq \perp$, then for every $\mathsf{hint} \leftarrow \mathsf{HintGen}(\mathsf{ek}, \mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]})$,

$$\mathsf{HintDec}\left(\mathsf{ek}, \mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]}, \mathsf{hint}\right) = \mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]}).$$

For soundness, we demand that it must be computationally infeasible to produce a hint and partial decryptions that may cause HintDec to differ from an honest execution of Dec on the same public inputs. In fact, the adversary is given all secret keys and is allowed to produce all ciphertexts, partial decryptions and hints that will be used by HintDec and Dec.

Definition 9 (Helper Soundness)**.** *A helper-aided decryption scheme is* sound *if all non-uniform PPT adversaries $\mathcal{A}$ have a negligible probability of winning the following game:*

- *The challenger runs $(\mathsf{ek}, \mathsf{sk}_1, \ldots, \mathsf{sk}_N, \mathsf{dk}) \leftarrow \mathsf{Setup}(1^\lambda, 1^B, 1^N, 1^t)$ and sends everything (including all secret keys) to $\mathcal{A}$.*
- *$\mathcal{A}$ outputs an admissible batch $\{\mathsf{ct}_i\}_{i\in[B]}$, a subset $T \subseteq [N]$ with $|T| \ge t$, partial decryptions $\{\mathsf{pd}_j\}_{j\in T}$, and a hint* hint*.*
- *If $\mathsf{Verify}(\mathsf{dk}, j, \mathsf{pd}_j, \{\mathsf{ct}_i\}_{i\in[B]}) = 0$ for some $j \in T$, or $\mathsf{pd} := \mathsf{Combine}(\mathsf{dk}, T, \{\mathsf{pd}_j\}_{j\in T}) = \perp$, the adversary loses.*
- *$\mathcal{A}$ wins if*

$$\mathsf{HintDec}(\mathsf{ek}, \mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]}, \mathsf{hint}) \notin \left\{\perp,\ \mathsf{Dec}(\mathsf{dk}, \mathsf{pd}, \{\mathsf{ct}_i\}_{i\in[B]})\right\}.$$

7.2 Simple BTE as Witness Encryption

At first glance, one might try to interpret the ciphertext $\mathsf{ct}_i = \left([k_i]_1,\ \mathsf{msg}_i + [k_i\tau^{B+1}]_T\right)$ as a witness encryption for the relation:

$$[1]_1 \circ w = [\tau^{B+1}]_T$$

Although this yields the appropriate ciphertext, the decryptor will never be able to produce the valid witness $[\tau^{B+1}]_2$ for this relation (even given the partial decryptions). Instead, consider the following relation $\mathcal{R}_j$ defined as:

$$\mathcal{R}_j : \begin{bmatrix} [1]_1 & 0 & \cdots & 0 & [\tau]_1 \\ 0 & [1]_1 & \cdots & 0 & [\tau^2]_1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & [1]_1 & [\tau^B]_1 \end{bmatrix} \circ \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_B \\ w_{B+1} \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ [\tau^{B+1}]_T \\ \vdots \\ 0 \end{bmatrix},$$

where $[\tau^{B+1}]_T$ appears as the non-zero entry in the j-th row. The ciphertexts in the simple-BTE scheme can then be viewed as the ciphertext produced by a two-round *distributed* protocol that emulates the generic encryption algorithm above for the relations $\{\mathcal{R}_i\}_{i\in[B]}$. We now make the connection more explicit.

Observe that by running the generic encryption algorithm with randomness $(k_1, \ldots, k_B)$ for the relation $\mathcal{R}_j$, the ciphertext produced is of the form:

$$\left([k_1]_1, [k_2]_1, \ldots, [k_B]_1, \left[\sum_{i=1}^{B} k_i \cdot \tau^i\right]_1,\ [k_j \cdot \tau^{B+1}]_T + \mathsf{msg}_j\right)$$

As it turns out, if we want to encrypt different messages $\{\mathsf{msg}_j\}_{j\in[B]}$ under relations $\{\mathcal{R}_j\}_{j\in[B]}$, respectively, we can *securely* share the randomness $(k_1, \ldots, k_B)$ between the ciphertexts. This leads to a compression in the ciphertext size from $B(B+1) \times \mathbb{G}_1 + B \times \mathbb{G}_T$ group elements to $(B+1) \times \mathbb{G}_1 + B \times \mathbb{G}_T$ group elements.[4] The ciphertext then has the form:

$$\mathsf{ct} = \begin{pmatrix} [k_1]_1, [k_2]_1, \ldots, [k_B]_1, \left[\sum_{i=1}^{B} k_i \cdot \tau^i\right]_1, \\ [k_1 \cdot \tau^{B+1}]_T + \mathsf{msg}_1, \ldots, [k_B \cdot \tau^{B+1}]_T + \mathsf{msg}_B \end{pmatrix}.$$

This is exactly the batch of ciphertexts together with the partial decryption in the simple-BTE scheme.

- The user who creates the i-th ciphertext in the batch samples k_i locally and publishes the ciphertext

  $$\mathsf{ct}_i = \left([k_i]_1,\ \mathsf{msg}_i + [k_i\tau^{B+1}]_T\right)$$

  where the first component of ct_i corresponds to the i-th entry of ct and the second component corresponds to the $(B+1+i)$-th entry of ct.

- The committee publishes the partial decryption $\mathsf{pd} = \left[\sum_{i=1}^{B} k_i \cdot \tau^i\right]_1$, which corresponds to the $(B+1)$-th component of ct.

Thus the users and the committee jointly produce the ciphertext ct in a distributed manner. Moreover, the decryption key dk of the simple-BTE scheme readily contains the witness required to decrypt the ciphertext.

[4] A similar strategy was used in [BCF+25] to compress the ciphertext size of the batched threshold encryption scheme with silent setup.

7.3 Randomness Recovery for Simple BTE

We now apply the randomness-recovery transform directly to the Simple BTE ciphertext from Fig. 1. Here we take the recovered key space to be $\mathbb{G}_T$ itself. Let $H_R : \mathbb{G}_T \times \{0,1\}^{\ell_m} \to \mathbb{F}_p$ and $H_M : \mathbb{G}_T \to \{0,1\}^{\ell_m}$ be random oracles. For a message $m \in \{0,1\}^{\ell_m}$, encryption samples $K \leftarrow \mathbb{G}_T$, computes $k := H_R(K, m)$, and outputs

$$\mathsf{ct} := \left([k]_1,\ k \cdot E + K,\ H_M(K) \oplus m\right),$$

where $E = \left[\tau^{B+1}\right]_T$ is parsed from ek. The decryption algorithm Dec of the transformed scheme computes the mask P_i from the decryption equation as before, recovers $K_i := \mathsf{ct}_{i,2} - P_i$, $m_i := \mathsf{ct}_{i,3} \oplus H_M(K_i)$, and $k_i := H_R(K_i, m_i)$, and outputs m_i if the re-encryption check $\mathsf{ct}_{i,1} = [k_i]_1$ passes and $\perp$ for that slot otherwise.

Given a combined partial decryption pd and a batch $\{\mathsf{ct}_i\}_{i\in[B]}$, the helper computes

$$P_i := \mathsf{pd} \circ h_{B+1-i} - \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i}.$$

For well-formed ciphertexts, $P_i = \left[k_i \tau^{B+1}\right]_T$. The helper then recovers

$$K_i := \mathsf{ct}_{i,2} - P_i, \qquad m_i := \mathsf{ct}_{i,3} \oplus H_M(K_i), \qquad k_i := H_R(K_i, m_i).$$

Assume for now that all ciphertexts are well-formed and can be decrypted successfully; we will handle malformed ciphertexts later. The helper can now choose between two possible hint formats.

Bandwidth-optimized. In the bandwidth-optimized variant, the helper sends $\{k_i\}_{i\in[B]}$. The verifier computes

$$P_i := k_i \cdot E, \qquad K_i := \mathsf{ct}_{i,2} - P_i, \qquad m_i := \mathsf{ct}_{i,3} \oplus H_M(K_i)$$

for each $i \in [B]$, and checks that $k_i = H_R(K_i, m_i)$. It then samples $r \leftarrow \mathbb{F}_p^B$ and checks

$$\sum_{i\in[B]} r_i \cdot \mathsf{ct}_{i,1} \stackrel{?}{=} \left(\sum_{i\in[B]} r_i k_i\right) \cdot g_1.$$

The verification cost is one $\mathbb{G}_1$-MSM, B scalar multiplications in $\mathbb{G}_T$ to compute the P_i values, and $O(B)$ hash evaluations and field operations. Note that this is still an asymptotic improvement over carrying out the decryption, which requires $O(B \log B)$ group operations. As in the generic randomness-recovery transform, the hint can be compressed further to 16 bytes by hashing to a PRG seed that is expanded to the field element.

Verification-optimized. In the verification-optimized variant, the helper sends $\{K_i\}_{i\in[B]}$. The verifier computes

$$m_i := \mathsf{ct}_{i,3} \oplus H_M(K_i), \qquad k_i := H_R(K_i, m_i), \qquad P_i := \mathsf{ct}_{i,2} - K_i$$

for each $i \in [B]$. It then samples $r \leftarrow \mathbb{F}_p^B$ and checks

$$\sum_{i\in[B]} r_i \cdot \mathsf{ct}_{i,1} \stackrel{?}{=} \left(\sum_{i\in[B]} r_i k_i\right) \cdot g_1$$

and

$$\sum_{i\in[B]} r_i \cdot P_i \stackrel{?}{=} \left(\sum_{i\in[B]} r_i k_i\right) \cdot E.$$

The verification cost is one $\mathbb{G}_1$-MSM, one $\mathbb{G}_T$-MSM, and $O(B)$ hash evaluations and field operations. The hints are larger, since each K_i is a target-group element, but the verifier avoids the B independent target-group scalar multiplications from the field-element variant. In both variants, the verifier performs no pairings.

7.4 Certifying Malformed Ciphertexts

The fast path above assumes that the helper can recover consistent randomness for every ciphertext. When a ciphertext is malformed, this is no longer true: the helper can provide a claimed mask P_i, but the verifier must still be convinced that this value came from the real BTE decryption equation rather than from a helper trying to suppress a valid ciphertext. The generic fallback described above would run the witness-encryption decryption algorithm locally, but for the simple-BTE relation that costs $O(B)$ pairings per ciphertext. If many ciphertexts are malformed, this destroys the benefit of helper-aided verification.

AFGHO commitments. We use the structure-preserving commitments of Abe et al. [AFG+10], which allow committing to a *vector of group elements* with a single target-group element. A commitment key for $\mathbb{G}_1$-vectors of length n is a vector $v \in \mathbb{G}_2^n$, and the commitment to $A \in \mathbb{G}_1^n$ is

$$\mathsf{Com}_v(A) := \sum_{\ell=1}^{n} A_\ell \circ v_\ell \in \mathbb{G}_T.$$

Commitments for $\mathbb{G}_2$-vectors are defined symmetrically: for a key $w \in \mathbb{G}_1^n$, the commitment to $B \in \mathbb{G}_2^n$ is $\mathsf{Com}_w(B) := \sum_{\ell=1}^{n} w_\ell \circ B_\ell$. The commitment is *binding* under the SXDH assumption for uniformly random commitment keys [AFG+10], and under the q-Auxiliary Structured Double Pairing (q-ASDBP) assumption [BMM+21] for the structured keys used in the TIPP and MIPP arguments.

The goal, then, is to certify only the expensive cross-terms in the BTE decryption equation. We do this with Inner Pairing Product Proofs from [BMM+21], which constructs arguments for the following relations (witness elements shown in red):

$$\mathcal{R}_{\mathsf{TIPP}} = \left\{ (T, U, Z, v, w\,;\, A \in \mathbb{G}_1^n,\, B \in \mathbb{G}_2^n) \;\middle|\; \begin{array}{l} T = \sum_{\ell=1}^{n} A_\ell \circ v_\ell, \\ U = \sum_{\ell=1}^{n} w_\ell \circ B_\ell, \\ Z = \sum_{\ell=1}^{n} A_\ell \circ B_\ell \end{array} \right\}$$

$$\mathcal{R}_{\mathsf{MIPP}} = \left\{ (T, C, v, \beta\,;\, A \in \mathbb{G}_1^n) \;\middle|\; \begin{array}{l} T = \sum_{\ell=1}^{n} A_\ell \circ v_\ell, \\ C = \sum_{\ell=1}^{n} \beta^{\ell-1} \cdot A_\ell \end{array} \right\}$$

Both instantiations have logarithmic proof size and verification time and can be viewed as commit-and-prove style proof systems over AFGHO-commitments [AFG+10].

Let $M \subseteq [B]$ be the set of ciphertexts that the helper claims are malformed. The helper will provide $\{P_i\}_{i \in M}$ to the verifier together with a proof that they have been computed correctly. We start with a single flagged ciphertext ct_j. The helper's claimed mask P_j for this ciphertext satisfies the decryption equation:

$$P_j = \mathsf{pd} \circ h_{B+1-j} - \sum_{\ell \neq j} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-j}.$$

Rearranging gives

$$P_j + \underbrace{\sum_{\ell=1}^{B} \mathsf{ct}_{\ell,1} \circ S_\ell^{(j)}}_{Z_j} = \mathsf{pd} \circ h_{B+1-j},$$

where

$$S_\ell^{(j)} := \begin{cases} 0 & \text{if } \ell = j, \\ h_{\ell+B+1-j} & \text{if } \ell \neq j. \end{cases}$$

Thus, for one malformed ciphertext, certifying P_j reduces to certifying the single cross-term Z_j. For a general malformed set M, we can aggregate the same equations with a fresh random vector $s = (s_i)_{i \in M} \leftarrow\$\ \mathbb{F}_p^{|M|}$ and define

$$R_M := \sum_{i \in M} s_i \cdot h_{B+1-i}$$

and, for each $\ell \in [B]$,

$$S_\ell^{(M)} := \sum_{\substack{i \in M \\ i \neq \ell}} s_i \cdot h_{\ell+B+1-i}.$$

Taking the same random linear combination of the decryption equations yields

$$\sum_{i \in M} s_i \cdot P_i + \underbrace{\sum_{\ell=1}^{B} \mathsf{ct}_{\ell,1} \circ S_\ell^{(M)}}_{Z_M} = \mathsf{pd} \circ R_M.$$

Thus verifying that $\{P_i\}_{i \in M}$ was computed correctly reduces to verifying that Z_M was computed correctly and checking the aggregated decryption equation above. We now describe how to efficiently generate a proof for Z_M. Observe that if we had AFGHO-commitments [AFG+10] to the vectors $\{\mathsf{ct}_{\ell,1}\}_{\ell \in [B]}$ and $\{S_\ell^{(M)}\}_{\ell \in [B]}$,

$$U_M = \sum_{\ell=1}^{B} w_\ell \circ S_\ell^{(M)}$$

$$T = \sum_{\ell=1}^{B} \mathsf{ct}_{\ell,1} \circ v_\ell$$

then the helper can simply generate a proof for $\mathcal{R}_{\mathsf{TIPP}}$ and the verifier can check that $Z_M = \sum_{\ell=1}^{B} \mathsf{ct}_{\ell,1} \circ S_\ell^{(M)}$ in logarithmic time. However, the verifier still needs to create the commitments to the witness vectors. Naively, this would cost $O(B|M|)$ operations to even compute the $S_\ell^{(M)}$ elements and an additional $O(B)$ pairings to compute the corresponding AFGHO-commitments. Instead, the verifier can be given

$$U^{(i)} := \sum_{\ell \neq i} w_\ell \circ h_{\ell+B+1-i} \in \mathbb{G}_T$$

for each $i \in [B]$ during the setup phase. The verifier can then compute

$$U_M = \sum_{i \in M} s_i \cdot U^{(i)}$$

using an MSM of size $|M|$ in $\mathbb{G}_T$. We can also avoid the B pairings when computing T by having the helper send T and prove, with an MIPP proof, that it is the AFGHO commitment to the public ciphertext vector $(\mathsf{ct}_{\ell,1})_{\ell \in [B]}$. Concretely, after T is fixed, the verifier samples a fresh scalar $\beta \leftarrow\!\!\$\, \mathbb{F}_p$, computes

$$C = \sum_{\ell=1}^{B} \beta^{\ell-1} \cdot \mathsf{ct}_{\ell,1},$$

and checks an MIPP proof for

$$T = \sum_{\ell=1}^{B} \mathsf{ct}_{\ell,1} \circ v_\ell, \qquad C = \sum_{\ell=1}^{B} \beta^{\ell-1} \cdot \mathsf{ct}_{\ell,1}.$$

Since the AFGHO commitment is binding, the vector A committed in T is fixed before β is sampled. If $A \neq (\mathsf{ct}_{\ell,1})_{\ell \in [B]}$, then $\sum_{\ell=1}^{B} \beta^{\ell-1} \cdot (A_\ell - \mathsf{ct}_{\ell,1}) = 0$ requires β to be a root of a nonzero polynomial of degree at most $B-1$, which happens with probability at most $(B-1)/p$ by the Schwartz-Zippel lemma. Thus the verifier replaces the direct B-pairing computation of T with one $\mathbb{G}_1$-MSM of size B and logarithmic MIPP verification, at the cost of an additional $(B-1)/p$ soundness error.

Although the verifier does not need to materialize the $\{S_\ell^{(M)}\}_{\ell\in[B]}$ vector, the prover requires it to generate the TIPP proof. We now describe how to compute the $\{S_\ell^{(M)}\}_{\ell\in[B]}$ vector in $O(B\log B)$ group operations instead of $O(|M|B)$. If we extend the challenge vector by setting $s_i := 0$ for every $i \notin M$, and also set $h_{B+1} := 0$, we get

$$S_\ell^{(M)} = \sum_{i=1}^{B} s_i \cdot h_{\ell+B+1-i}.$$

Thus all B values $\{S_\ell^{(M)}\}_{\ell\in[B]}$ can be computed together by an FFT-style convolution in $O(B\log B)$ group operations in $\mathbb{G}_2$. Once the MIPP and TIPP proofs verify, the verifier performs the same randomness-recovery check used in the fast path to confirm that each flagged ciphertext is indeed malformed.

7.5 The Full Protocol

Putting the pieces together, we obtain the following protocol. For clarity of exposition, we present it as a *public-coin interactive protocol* between a helper $\mathcal{H}$ and a verifier $\mathcal{V}$. It can be compiled into the non-interactive algorithms $(\mathsf{HintGen}, \mathsf{HintDec})$ of Section 7.1 in a straightforward manner using the Fiat-Shamir transform.

Common input. The encryption key ek (parsed to recover $E = \left[\tau^{B+1}\right]_T$), the decryption key dk — augmented with the AFGHO commitment keys $v \in \mathbb{G}_2^B$, $w \in \mathbb{G}_1^B$, the SRS of the MIPP/TIPP arguments [BMM+21], and the precomputed values $\{U^{(i)}\}_{i\in[B]}$ — the combined partial decryption pd, and a batch of transformed ciphertexts $\{\mathsf{ct}_i\}_{i\in[B]}$. The helper chooses one of the two hint variants: bandwidth-optimized or verification-optimized.

Round 1 ($\mathcal{H}$).

- For $i \in [B]$, compute
$$P_i := \mathsf{pd} \circ h_{B+1-i} - \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i}.$$
- For each $i \in [B]$, compute $K_i := \mathsf{ct}_{i,2} - P_i$, $m_i := \mathsf{ct}_{i,3} \oplus H_M(K_i)$, $k_i := H_R(K_i, m_i)$.
- Set $M := \{i \in [B] : \mathsf{ct}_{i,1} \neq [k_i]_1\}$ and compute $T := \sum_{\ell=1}^{B} \mathsf{ct}_{\ell,1} \circ v_\ell$.
- Send M, $\{P_i\}_{i\in M}$, T, and
  - $\{k_i\}_{i\notin M}$ if bandwidth-optimized, or
  - $\{K_i\}_{i\notin M}$ if verification-optimized.

If $M = \emptyset$, rounds 2–5 are skipped.

Round 2 ($\mathcal{V}$). Sample and send $\beta \leftarrow\!\$\, \mathbb{F}_p$.

Round 3 ($\mathcal{H}$). Send an MIPP proof π_{MIPP} for the statement (T, C, v, β), where $C := \sum_{\ell=1}^{B} \beta^{\ell-1} \cdot \mathsf{ct}_{\ell,1}$.

Round 4 ($\mathcal{V}$). Sample and send $s = (s_i)_{i\in M} \leftarrow\!\$\, \mathbb{F}_p^{|M|}$.

Round 5 ($\mathcal{H}$). Compute $\{S_\ell^{(M)}\}_{\ell\in[B]}$ via the FFT-style convolution described above, and send $Z_M := \sum_{\ell=1}^{B} \mathsf{ct}_{\ell,1} \circ S_\ell^{(M)}$ together with a TIPP proof π_{TIPP} for the statement (T, U_M, Z_M).

Verification ($\mathcal{V}$). Output $\perp$ (reject) if any check below fails.

- *Well-formed slots.* For each $i \notin M$:
  - if bandwidth-optimized: compute $P_i := k_i \cdot E$, $K_i := \mathsf{ct}_{i,2} - P_i$, $m_i := \mathsf{ct}_{i,3} \oplus H_M(K_i)$, and check $k_i = H_R(K_i, m_i)$;

– if verification-optimized: compute $m_i := \mathsf{ct}_{i,3} \oplus H_M(K_i)$, $k_i := H_R(K_i, m_i)$, $P_i := \mathsf{ct}_{i,2} - K_i$.

Sample $r = (r_i)_{i\in[B]\setminus M} \leftarrow\$ \mathbb{F}_p^{|[B]\setminus M|}$ and check

$$\sum_{i\notin M} r_i \cdot \mathsf{ct}_{i,1} = \Big(\sum_{i\notin M} r_i k_i\Big) \cdot g_1.$$

If verification-optimized, additionally check $\sum_{i\notin M} r_i \cdot P_i = \big(\sum_{i\notin M} r_i k_i\big) \cdot E$.

- *Flagged slots* (skipped if $M = \emptyset$):
  - Compute $C := \sum_{\ell=1}^{B} \beta^{\ell-1} \cdot \mathsf{ct}_{\ell,1}$ and verify π_{MIPP} for (T, C, v, β).
  - Compute $R_M := \sum_{i\in M} s_i \cdot h_{B+1-i}$ and $U_M := \sum_{i\in M} s_i \cdot U^{(i)}$, and verify π_{TIPP} for (T, U_M, Z_M).
  - Check $\sum_{i\in M} s_i \cdot P_i + Z_M = \mathsf{pd} \circ R_M$.
  - For each $i \in M$: compute $K_i := \mathsf{ct}_{i,2} - P_i$, $m_i := \mathsf{ct}_{i,3} \oplus H_M(K_i)$, $k_i' := H_R(K_i, m_i)$, and check $\mathsf{ct}_{i,1} \neq [k_i']_1$.
- Output m_i for each $i \notin M$ and $\perp$ for each $i \in M$.

Theorem 8. *Assuming the B-ASDBP assumption [BMM+21] (in both its $\mathbb{G}_1$ and $\mathbb{G}_2$ variants), the MIPP and TIPP arguments of [BMM+21] satisfy completeness and knowledge soundness, and H_R, H_M are modeled as random oracles, the protocol above satisfies completeness (Definition 8) and helper soundness (Definition 9).*

Proof. We first prove completeness. Let $\kappa_\ell := \mathsf{dlog}_{g_1}(\mathsf{ct}_{\ell,1})$ and since every share pd_j passes Verify, the proof of Theorem 1 shows that each pd_j must equal the honest partial decryption. Hence, we have

$$\mathsf{pd} = \Big[\textstyle\sum_{\ell\in[B]} \kappa_\ell \tau^\ell\Big]_1, \qquad \text{so} \qquad P_i = \mathsf{pd} \circ h_{B+1-i} - \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i} = \big[\kappa_i \tau^{B+1}\big]_T$$

for every $i \in [B]$. An honest helper faithfully follows Dec to compute the masks P_i, so the derived values (K_i, m_i, k_i) in round 1 coincide with those computed by Dec, and the helper's flagged set M is exactly the set of slots where Dec outputs $\perp$. For $i \notin M$ we have $\mathsf{ct}_{i,1} = [k_i]_1$, i.e., $k_i = \kappa_i$, and thus $P_i = k_i \cdot E$, so the verifier's checks succeed, and the verifier outputs the same m_i as Dec. For $i \in M$, the MIPP and TIPP statements are true, so the honest proofs verify by completeness and the aggregated equation $\sum_{i\in M} s_i \cdot P_i + Z_M = \mathsf{pd} \circ R_M$ holds. The final re-encryption check confirms $\mathsf{ct}_{i,1} \neq [k_i]_1$, which holds exactly because the helper flagged slot i. Thus the verifier outputs $\perp$ for $i \in M$, identically to Dec.

For soundness, we first argue that the commitments used by MIPP and TIPP are binding. The commitment keys in the SRS of [BMM+21] are structured as even powers of the trapdoors, $v = (\big[\gamma^{2(\ell-1)}\big]_2)_{\ell\in[B]}$ and $w = (\big[\eta^{2(\ell-1)}\big]_1)_{\ell\in[B]}$ for trapdoors $\gamma, \eta \leftarrow\$ \mathbb{F}_p$ sampled at setup. Suppose an adversary produces $A \neq A'$ with $\mathsf{Com}_v(A) = \mathsf{Com}_v(A')$. Then

$$\sum_{\ell=1}^{B} (A_\ell - A'_\ell) \circ v_\ell = 0_{\mathbb{G}_T} \quad \text{with} \quad A - A' \neq 0_{\mathbb{G}_1^B},$$

which is exactly a solution to B-ASDBP in its $\mathbb{G}_2$ variant. The symmetric argument for w keys yields a solution to the $\mathbb{G}_1$ variant. Hence, except with negligible probability, the vectors extracted by the MIPP and TIPP knowledge extractors from valid proofs for T and U_M equal the intended vectors $(\mathsf{ct}_{\ell,1})_{\ell\in[B]}$ and $(S_\ell^{(M)})_{\ell\in[B]}$.

Now suppose the verifier accepts. For each $i \in [B]$ let

$$P_i^* := \mathsf{pd} \circ h_{B+1-i} - \sum_{\substack{\ell\in[B]\\ \ell\neq i}} \mathsf{ct}_{\ell,1} \circ h_{\ell+B+1-i}$$

denote the true mask, i.e., the value that Dec computes for slot i. Exactly as in the completeness argument, the Verify checks force $\mathsf{pd} = \left[\sum_{\ell\in[B]} \kappa_\ell \tau^\ell\right]_1$ and hence $P_i^* = \left[\kappa_i \tau^{B+1}\right]_T = \kappa_i \cdot E$ for every $i \in [B]$ where $[\kappa_i]_1 = \mathsf{ct}_{i,1}$.

Well-formed slots ($i \notin M$). Here the hint contains *claimed* scalars k_i (sent directly in the bandwidth-optimized variant, or derived from the claimed K_i in the verification-optimized variant). In the bandwidth-optimized variant the verifier checks the random linear combination $\sum_{i\notin M} r_i \cdot \mathsf{ct}_{i,1} = (\sum_{i\notin M} r_i k_i) \cdot g_1$ with r sampled after the k_i are fixed. If $\mathsf{ct}_{i,1} \neq [k_i]_1$ for some $i \notin M$, the check passes with probability at most $1/p$. Excluding this event, $k_i = \kappa_i$ for all $i \notin M$, so the verifier's mask $P_i = k_i \cdot E$ equals P_i^*; hence K_i, m_i, and the re-encryption check computed by the verifier coincide with those computed by Dec, and the outputs agree. In the verification-optimized variant the same argument applies to the two random-linear-combination checks where the first forces $\mathsf{ct}_{i,1} = [k_i]_1$, i.e., $k_i = \kappa_i$, and the second forces $\mathsf{ct}_{i,2} - K_i = k_i \cdot E = P_i^*$ for all $i \notin M$, after which the verifier's values again coincide with those of Dec.

Flagged slots ($i \in M$). By knowledge soundness and binding, the MIPP proof forces $T = \mathsf{Com}_v((\mathsf{ct}_{\ell,1})_\ell)$ except with probability $(B-1)/p$, since β is sampled after T is fixed. The verifier computes $U_M = \sum_{i\in M} s_i \cdot U^{(i)}$ itself, which by linearity equals $\mathsf{Com}_w((S_\ell^{(M)})_\ell)$. The TIPP proof then forces $Z_M = \sum_{\ell\in[B]} \mathsf{ct}_{\ell,1} \circ S_\ell^{(M)}$. The true masks satisfy $\sum_{i\in M} s_i \cdot P_i^* + Z_M = \mathsf{pd} \circ R_M$ by construction, and the verifier's check on the claimed masks shows that

$$\sum_{i\in M} s_i \cdot P_i + Z_M = \mathsf{pd} \circ R_M.$$

Thus, $P_i = P_i^*$ for all $i \in M$ except with probability $1/p$, as s is sampled after the P_i are fixed. The final re-encryption check therefore recomputes exactly the values of Dec and confirms $\mathsf{ct}_{i,1} \neq [k'_i]_1$, i.e., Dec outputs $\perp$ for slot i — matching the verifier's output.

A union bound over the failure events shows that the total failure probability is negligible, completing the proof. □

8 Implementation and Benchmarks

To evaluate the concrete performance of our scheme, we implemented it in Rust, using arkworks [ac22] for implementations of pairing-friendly curves. Our implementation can be found at https://github.com/commonwarexyz/simple-bte.

Setup. Experiments were run on an M5 MacBook Pro with 48 GB of RAM in single-threaded mode. We use BLS12-381 [BLS03] as our pairing-friendly curve. The compressed representation of $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ elements is 48, 96, and 576 bytes respectively. Group exponentiations in $\mathbb{G}_1$ and $\mathbb{G}_2$ take approximately 68 μs and 193 μs respectively, and a pairing takes approximately 443 μs.

Benchmarks. We now discuss microbenchmarks for various algorithms in our scheme.

- **Encryption.** Always takes approximately 0.47 ms irrespective of batch size and the number of parties.
- **NIZK Verify.** Verifies that a batch of ciphertexts is valid using batch verification of Schnorr proofs [APB+04] which can be carried out using $O(B)$ hashes and an MSM of size $O(B)$.
- **PartialDec.** Computes the partial decryption share for a given party using an MSM of size $O(B)$.
- **Verify.** Verifies a partial decryption share, by computing a multi-pairing of size $O(B)$.
- **Combine.** Combines a set of partial decryption shares into a final decryption by computing Lagrange coefficients and using an MSM of size $O(N)$.

- **Decryption.** Decrypts a batch of ciphertexts using a $\mathbb{G}_1$-FFT of size $O(B)$, a $\mathbb{G}_T$-FFT of size $O(B)$, and $O(B)$ pairings.
- **Batch Verification.** Verifies helper-provided decryption hints using MSMs and hashes, avoiding pairings on the verifier side.

Operation	$B = 8$	$B = 32$	$B = 128$	$B = 512$	$B = 2048$
PartialDec (ms)	0.832	2.208	5.963	18.40	59.55
Verify (ms)	1.508	4.074	14.61	55.87	222.66
Combine (ms)	0.678	0.678	0.681	0.676	0.677

Comparison with prior work. We compare our batch decryption times against three prior schemes: [ABD+25], which achieves $O(B \log B)$ decryption but lacks censorship resistance, TrX [FPTX25], which offers censorship resistance but has $O(B \log^2 B)$ decryption complexity, and the $\mathsf{BE}_{\mathsf{short}}$ construction from [BNRT26]. The [ABD+25] numbers are the core protocol times from their Table 3 (unweighted, NIZK overhead excluded). The [FPTX25] numbers are from benchmarking their implementation (Digest, ComputeEvalProofs, and Decrypt) on our machine, using the same curve and single-threaded setup. The [BNRT26] numbers are from benchmarking their open source implementation on our machine.

Batch size B	Ours	TrX [FPTX25]	[ABD+25]	[BNRT26]
32	121.50 ms	81.91 ms	301 ms	150.48 ms
128	593.63 ms	504.61 ms	1.4 s	715.30 ms
512	2.80 s	4.20 s	6.6 s	3.27 s
2048	12.91 s	43.36 s	28.4 s	14.78 s

At small batch sizes ($B \leq 128$), TrX [FPTX25] is concretely faster, but our scheme is the fastest of all schemes at $B \geq 512$, with the gap widening as the batch size grows.

Pipelined decryption breakdown. As discussed in Section 6.2, the decryption algorithm can be split into a pre-decryption phase (independent of the combined partial decryption) and a finalization phase. The table below shows the cost of each phase. The pre-decryption phase dominates total decryption time, and can run concurrently with the threshold protocol (PartialDec, Verify, Combine). After the threshold protocol completes, only the finalization phase remains.

Phase	$B = 32$	$B = 128$	$B = 512$	$B = 2048$
Pre-decrypt (ms)	105.31	528.25	2542.16	11864.87
Finalize (ms)	16.19	65.38	262.41	1048.68
Total (ms)	121.50	593.63	2804.57	12913.55

For example, at $B = 2048$, the finalization phase takes only 1.05 s out of 12.91 s total — an $\approx 12\times$ reduction in latency from the time the threshold protocol completes to plaintext recovery.

Batch Verification. We also implemented the helper-aided batch verification procedure from Section 7.3. The helper first runs the expensive decryption procedure to produce hints; once these hints are available, the verifier checks the whole batch using MSMs and hashes without performing pairings. The speedups below compare verifier work against helper-side hint generation for the same batch size.

We also implemented the naive first attempt at batch verification from Section 7. As discussed in Section 7.2, even though the witness is readily available in the CRS, actually verifying the decryption equation Eq. (1) in Simple BTE requires $O(B)$ pairings. We implement the FFT optimization from Section 6.1 to reduce the cost to $O(B \log B)$ group operations and $O(B)$ pairings.

B	Helper	Naive	Bandwidth-opt.	Verification-opt.
32	121.61 ms	89.61 ms (1.4×)	11.53 ms (10.5×)	7.274 ms (16.7×)
128	596.13 ms	415.27 ms (1.4×)	45.48 ms (13.1×)	14.93 ms (39.9×)
512	2.84 s	1.927 s (1.5×)	179.46 ms (15.8×)	34.45 ms (82.4×)
2048	12.78 s	8.751 s (1.5×)	726.68 ms (17.6×)	112.01 ms (114.1×)

9 Acknowledgments

We thank Patrick O'Grady and Lucas Meier for their valuable feedback.

References

[ABD+25] Amit Agarwal, Kushal Babel, Sourav Das, Babak Poorebrahim Gilkalaye, Arup Mondal, Benny Pinkas, Peter Rindal, and Aayush Yadav. Weighted batched threshold encryption with applications to mempool privacy. Cryptology ePrint Archive, Paper 2025/2115, 2025. 1, 2, 3, 4, 12, 14, 35

[ac22] arkworks contributors. `arkworks` zksnark ecosystem. https://arkworks.rs, 2022. 4, 34

[ADG+26] Amit Agarwal, Sourav Das, Babak Poorebrahim Gilkalaye, Peter Rindal, and Victor Shoup. BTX: Simple and efficient batch threshold encryption. Cryptology ePrint Archive, Paper 2026/754, 2026. 4

[AFG+10] Masayuki Abe, Georg Fuchsbauer, Jens Groth, Kristiyan Haralambiev, and Miyako Ohkubo. Structure-preserving signatures and commitments to group elements. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 209–236. Springer, Berlin, Heidelberg, August 2010. 30, 31

[AFP25] Amit Agarwal, Rex Fernando, and Benny Pinkas. Efficiently-thresholdizable batched identity based encryption, with applications. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 69–100. Springer, Cham, August 2025. 1, 2, 3, 4, 12, 25, 27

[APB+04] Riza Aditya, Kun Peng, Colin Boyd, Ed Dawson, and Byoungcheon Lee. Batch verification for equality of discrete logarithms and threshold decryptions. In Markus Jakobsson, Moti Yung, and Jianying Zhou, editors, *ACNS 2004*, volume 3089 of *LNCS*, pages 494–508. Springer, Berlin, Heidelberg, June 2004. 34

[BCF+25] Jan Bormet, Arka Rai Choudhuri, Sebastian Faust, Sanjam Garg, Hussien Othman, Guru-Vamsi Policharla, Ziyan Qu, and Mingyuan Wang. BEAST-MEV: Batched threshold encryption with silent setup for MEV prevention. Cryptology ePrint Archive, Paper 2025/1419, 2025. 1, 2, 28

[BF01] Dan Boneh and Matthew K. Franklin. Identity-based encryption from the Weil pairing. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 213–229. Springer, Berlin, Heidelberg, August 2001. 3, 25

[BFOQ25] Jan Bormet, Sebastian Faust, Hussien Othman, and Ziyan Qu. BEAT-MEV: Epochless approach to batched threshold encryption for MEV prevention. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 3457–3476. USENIX Association, August 2025. 1, 2, 3, 4, 12

[BGW05] Dan Boneh, Craig Gentry, and Brent Waters. Collusion resistant broadcast encryption with short ciphertexts and private keys. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 258–275. Springer, Berlin, Heidelberg, August 2005. 3

[BLS03] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. Constructing elliptic curves with prescribed embedding degrees. In Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano, editors, *SCN 02*, volume 2576 of *LNCS*, pages 257–267. Springer, Berlin, Heidelberg, September 2003. 34

[BLT25] Dan Boneh, Evan Laufer, and Ertem Nusret Tas. Batch decryption without epochs and its application to encrypted mempools. Cryptology ePrint Archive, Report 2025/1254, 2025. 1

[BMM+21] Benedikt Bünz, Mary Maller, Pratyush Mishra, Nirvan Tyagi, and Psi Vesely. Proofs for inner pairing products and applications. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 65–97. Springer, Cham, December 2021. 30, 32, 33

[BNRT26] Dan Boneh, Rohit Nema, Arnab Roy, and Ertem Nusret Tas. Efficient batch threshold encryption using partial fraction techniques. Cryptology ePrint Archive, Paper 2026/674, 2026. 1, 2, 3, 4, 35

[BO22] Joseph Bebel and Dev Ojha. Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Report 2022/898, 2022. 1

[Boy08] Xavier Boyen. The uber-assumption family (invited talk). In Steven D. Galbraith and Kenneth G. Paterson, editors, *PAIRING 2008*, volume 5209 of *LNCS*, pages 39–56. Springer, Berlin, Heidelberg, September 2008. 39, 40

[BPR+08] Dan Boneh, Periklis A. Papakonstantinou, Charles Rackoff, Yevgeniy Vahlis, and Brent Waters. On the impossibility of basing identity based encryption on trapdoor permutations. In *49th FOCS*, pages 283–292. IEEE Computer Society Press, October 2008. 3

[CGPP24] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024. 1

[CGPW25] Arka Rai Choudhuri, Sanjam Garg, Guru-Vamsi Policharla, and Mingyuan Wang. Practical mempool privacy via one-time setup batched threshold encryption. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 3477–3495. USENIX Association, August 2025. 1, 2, 3, 4, 12, 25, 27

[Che52] Herman Chernoff. A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *The Annals of Mathematical Statistics*, pages 493–507, 1952. 22

[DF90] Yvo Desmedt and Yair Frankel. Threshold cryptosystems. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 307–315. Springer, New York, August 1990. 1

[DHMW23] Nico Döttling, Lucjan Hanzlik, Bernardo Magri, and Stella Wohnig. McFly: Verifiable encryption to the future made practical. In Foteini Baldimtsi and Christian Cachin, editors, *FC 2023, Part I*, volume 13950 of *LNCS*, pages 252–269. Springer, Cham, May 2023. 3, 25

[DJ01] Ivan Damgård and Mats Jurik. A generalisation, a simplification and some applications of Paillier's probabilistic public-key system. In Kwangjo Kim, editor, *PKC 2001*, volume 1992 of *LNCS*, pages 119–136. Springer, Berlin, Heidelberg, February 2001. 1

[ElG86] Taher ElGamal. On computing logarithms over finite fields. In Hugh C. Williams, editor, *CRYPTO'85*, volume 218 of *LNCS*, pages 396–402. Springer, Berlin, Heidelberg, August 1986. 1

[FGHP09] Anna Lisa Ferrara, Matthew Green, Susan Hohenberger, and Michael Østergaard Pedersen. Practical short signature batch verification. In Marc Fischlin, editor, *CT-RSA 2009*, volume 5473 of *LNCS*, pages 309–324. Springer, Berlin, Heidelberg, April 2009. 25, 26

[Fis05] Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *CRYPTO 2005*, volume 3621 of *LNCS*, pages 152–168. Springer, Berlin, Heidelberg, August 2005. 12

[FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018. 12

[FO98] Eiichiro Fujisaki and Tatsuaki Okamoto. A practical and provably secure scheme for publicly verifiable secret sharing and its applications. In Kaisa Nyberg, editor, *EUROCRYPT'98*, volume 1403 of *LNCS*, pages 32–46. Springer, Berlin, Heidelberg, May / June 1998. 26

[FO13] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013. 26

[FPS01] Pierre-Alain Fouque, Guillaume Poupard, and Jacques Stern. Sharing decryption in the context of voting or lotteries. In Yair Frankel, editor, *FC 2000*, volume 1962 of *LNCS*, pages 90–104. Springer, Berlin, Heidelberg, February 2001. 1

[FPTX25] Rex Fernando, Guru-Vamsi Policharla, Andrei Tonkikh, and Zhuolun Xiang. TrX: Encrypted mempools in high performance BFT protocols. Cryptology ePrint Archive, Report 2025/2032, 2025. 1, 2, 3, 4, 12, 35

[GHK+25] Sanjam Garg, Mohammad Hajiabadi, Dimitris Kolonelos, Abhiram Kothapalli, and Guru-Vamsi Policharla. A framework for witness encryption from linearly verifiable SNARKs and applications. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 504–539. Springer, Cham, August 2025. 2, 3, 25

[GJST81] Michael R Garey, David S. Johnson, Barbara B. Simons, and Robert Endre Tarjan. Scheduling unit–time tasks with arbitrary release times and deadlines. *SIAM Journal on Computing*, 10(2):256–269, 1981. 14, 15, 16

[GKPW24] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 352–386. Springer, Cham, August 2024. 1, 2, 3, 25

[GMR23] Nicolas Gailly, Kelsey Melissaris, and Yolan Romailler. tlock: Practical timelock encryption from threshold BLS. Cryptology ePrint Archive, Report 2023/189, 2023. 3, 25

[GWWW25] Junqing Gong, Brent Waters, Hoeteck Wee, and David J. Wu. Threshold batched identity-based encryption from pairings in the plain model. Cryptology ePrint Archive, Report 2025/2103, 2025. 1, 2, 3, 4

[Hal35] P. Hall. On representatives of subsets. *Journal of the London Mathematical Society*, s1-10(1):26–30, 1935. 21

[JNR25] Charanjit S. Jutla, Rohit Nema, and Arnab Roy. Partial fraction techniques for cryptography. Cryptology ePrint Archive, Report 2025/2081, 2025. 2

[KLJD23] Alireza Kavousi, Duc V. Le, Philipp Jovanovic, and George Danezis. BlindPerm: Efficient MEV mitigation with an encrypted mempool and permutation. Cryptology ePrint Archive, Report 2023/1061, 2023. 1

[KMW23] Dimitris Kolonelos, Giulio Malavolta, and Hoeteck Wee. Distributed broadcast encryption from bilinear groups. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part V*, volume 14442 of *LNCS*, pages 407–441. Springer, Singapore, December 2023. 3, 25

[Mau05] Ueli M. Maurer. Abstract models of computation in cryptography (invited paper). In Nigel P. Smart, editor, *10th IMA International Conference on Cryptography and Coding*, volume 3796 of *LNCS*, pages 1–12. Springer, Berlin, Heidelberg, December 2005. 2

[MS22] Dahlia Malkhi and Pawel Szalachowski. Maximal extractable value (mev) protection on a dag, 2022. 1

[Pai99] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 223–238. Springer, Berlin, Heidelberg, May 1999. 1

[PRV12] Periklis A. Papakonstantinou, Charles W. Rackoff, and Yevgeniy Vahlis. How powerful are the DDH hard groups? Cryptology ePrint Archive, Report 2012/653, 2012. 3

[RSA78] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the Association for Computing Machinery*, 21(2):120–126, February 1978. 1

[SAS24] Sora Suegami, Shinsaku Ashizawa, and Kyohei Shibano. Constant-cost batched partial decryption in threshold encryption. Cryptology ePrint Archive, Paper 2024/762, 2024. 1, 3, 4

[Sho97] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT'97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997. 2, 12

[SW05] Amit Sahai and Brent R. Waters. Fuzzy identity-based encryption. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 457–473. Springer, Berlin, Heidelberg, May 2005. 3, 25

A Hardness of Indexed (δ, q)-DBPDH in the GGM

We justify the indexed (δ, q)-DBPDH assumption from Definition 7. Since the instance contains inverse powers of α and negative powers of τ, we apply the rational-exponent extension of the Uber-assumption framework [Boy08].

Theorem 9. *The indexed (δ, q)-DBPDH assumption of Definition 7 holds in the Type-III generic bilinear group model.*

Proof. We express the assumption as a Decision (R, S, T, f)-Diffie-Hellman problem in the uber-assumption framework [Boy08]. Let X, Y, Z, and W denote formal variables corresponding to the secret exponents τ, α, β, and k respectively. The instance exposes the exponents

$$R = \langle 1,\ W \rangle \cup \left\langle X^i,\ Y^{-1}X^i,\ WY^{-1}X^i \right\rangle_{i \in [q]} \qquad \text{in } \mathbb{G}_1,$$

and

$$S = \langle 1,\ Z \rangle \cup \left\langle X^j \right\rangle_{j \in [2q] \setminus \{q+1\}} \cup \left\langle ZYX^d \right\rangle_{d \in D} \qquad \text{in } \mathbb{G}_2,$$

with $T = \langle 1 \rangle$, and the challenge polynomial is

$$f(X, Y, Z, W) = WX^{q+1}.$$

Rational exponents. The exponents $Y^{-1}X^i$ and $WY^{-1}X^i$ in R, and ZYX^d for negative $d \in D$ in S, are not polynomial but rational, so we apply the rational-exponent extension [Boy08, Section 6.2]: we multiply all exponents by $\Delta := YX^{\delta-1}$, the least common multiple of the denominators. Since τ, α are sampled from $\mathbb{F}_p^*$, the denominator Δ never vanishes and all instance elements are well-defined. Moreover, sampling τ, α, β from $\mathbb{F}_p^*$ instead of $\mathbb{F}_p$ as in the Master Theorem costs only a statistical distance of $3/p$.

Independence. In a Type-III bilinear context there is no computable isomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$, so f is dependent on $\langle R, S, T\rangle$ only if $f = \sum_{i,j} a_{i,j}\, r_i s_j + \sum_k b_k t_k$ for constants $a_{i,j}, b_k \in \mathbb{F}_p$. Every $r_i s_j$, every t_k, and f itself is a Laurent monomial, and distinct Laurent monomials are linearly independent, so it suffices to check that the monomial WX^{q+1} occurs neither in $T = \langle 1\rangle$ (immediate) nor among the products $r_i s_j$. Since no entry of S contains W, any product equal to WX^{q+1} must take its W-factor from R; we enumerate all products containing W:

$$
\begin{aligned}
&W \cdot 1 = W, \qquad W \cdot Z = WZ, \qquad W \cdot X^j = WX^j, \qquad W \cdot ZYX^d = WZYX^d,\\
&WY^{-1}X^i \cdot 1 = WY^{-1}X^i, \qquad WY^{-1}X^i \cdot Z = WZY^{-1}X^i,\\
&WY^{-1}X^i \cdot X^j = WY^{-1}X^{i+j}, \qquad WY^{-1}X^i \cdot ZYX^d = WZX^{i+d}.
\end{aligned}
$$

and observe that the challenge WX^{q+1} does not appear in any of these products. Hence f is independent of $\langle R, S, T\rangle$, and the Master Theorem [Boy08, Theorem 1] bounds the advantage of any generic adversary making polynomially many oracle queries by a negligible function, as all instance polynomials have degree $\mathsf{poly}(q)$. □

Corollary 2. *If the indexed (δ, q)-DBPDH assumption (Definition 7) holds, then the q-DBPDH assumption (Definition 6) holds.*

Proof. The indexed instance contains a superset of the terms of the q-DBPDH instance: dropping $\{[\alpha^{-1}\tau^i]_1\}_{i\in[q]}$, $\{[k\alpha^{-1}\tau^i]_1\}_{i\in[q]}$, $[\beta]_2$, and $\{[\beta\alpha\tau^i]_2\}_{i\in D}$ from an indexed instance yields a q-DBPDH instance with the same challenge, except that τ is sampled from $\mathbb{F}_p^*$ rather than $\mathbb{F}_p$, which changes the instance distribution by statistical distance at most $1/p$. Hence any distinguisher for q-DBPDH is also a distinguisher for indexed (δ, q)-DBPDH with advantage smaller by at most $1/p$. □